

What metrics do networked applications care about?

CS 456/656, Fall 2026

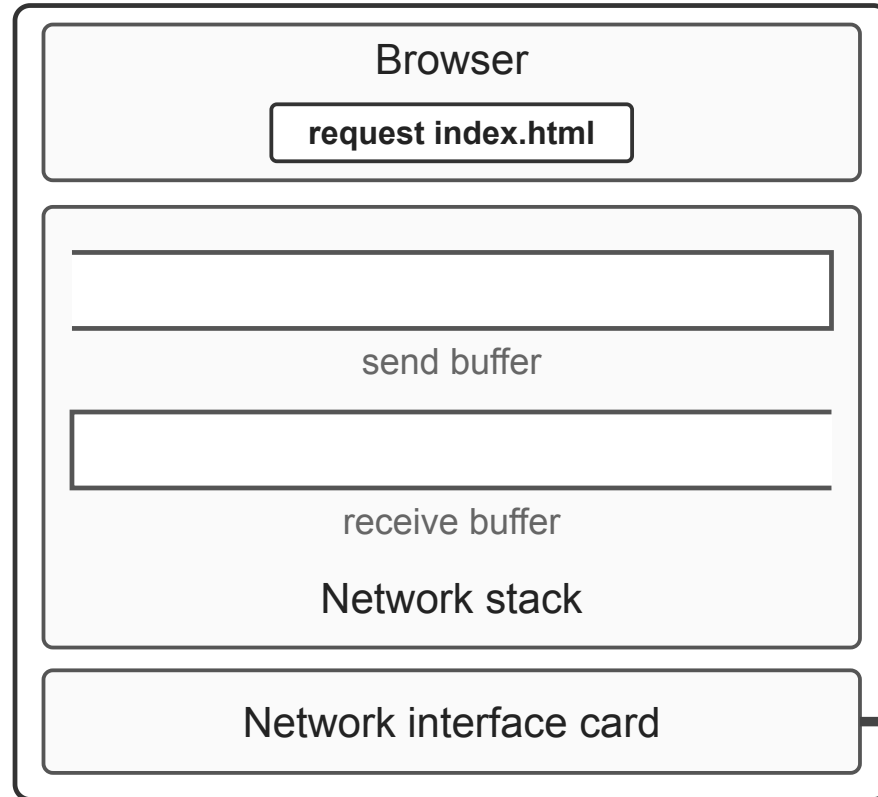
Mina Tahmasbi Arashloo

Outline

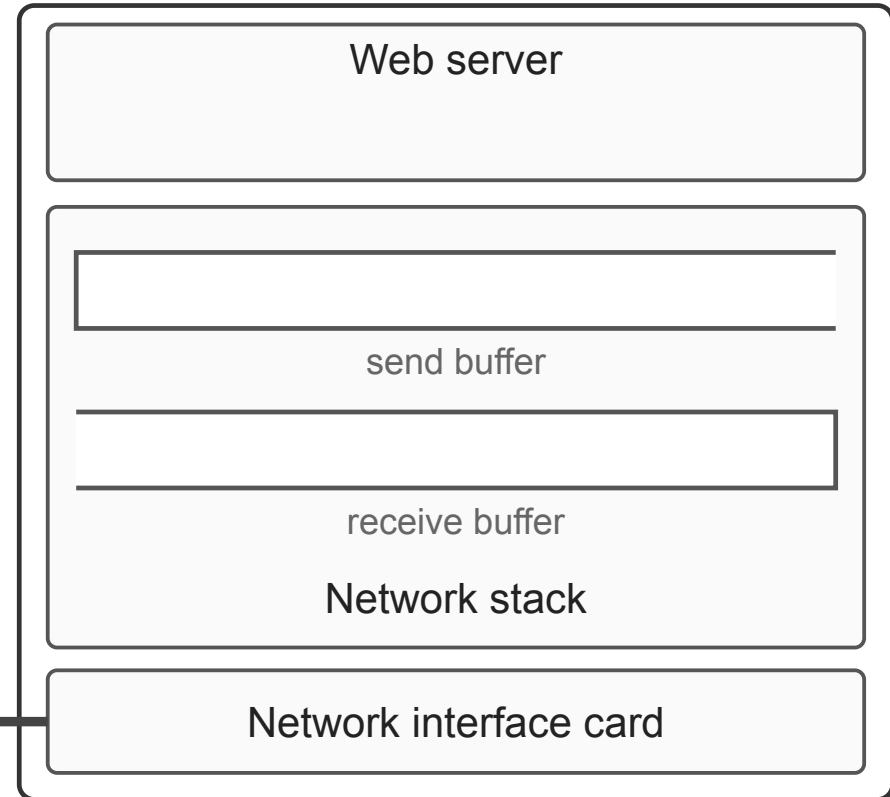
- **Where we were**
- Video streaming on a perfect network
- Video streaming on a real network
- What each application wants from the network
- Takeaways

Last time: we built a web browser

A: client

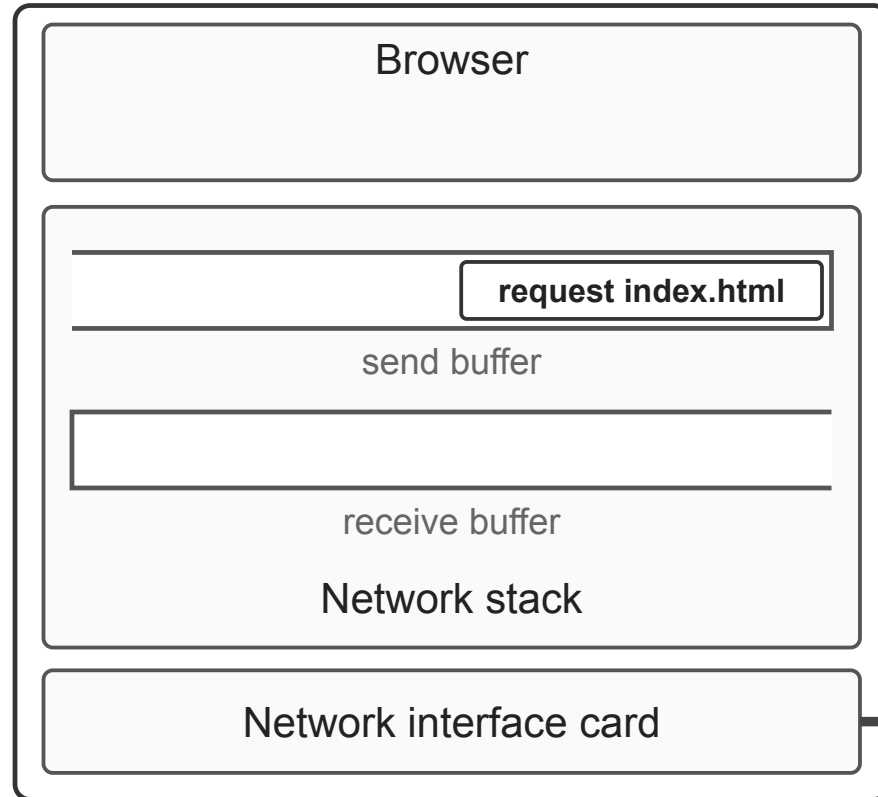


B: server

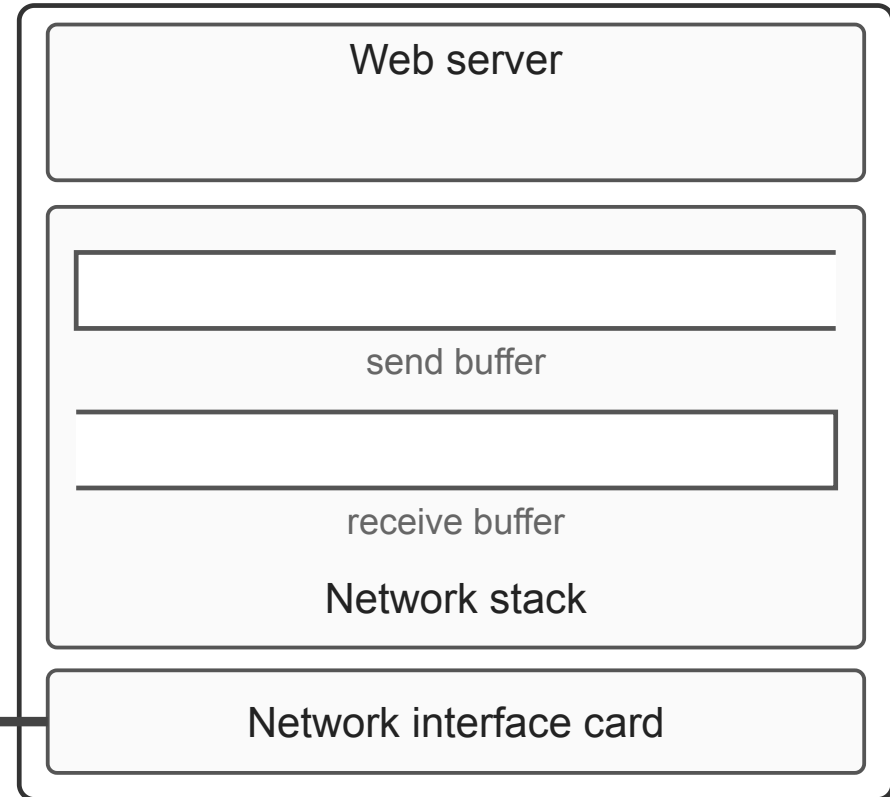


Last time: we built a web browser

A: client

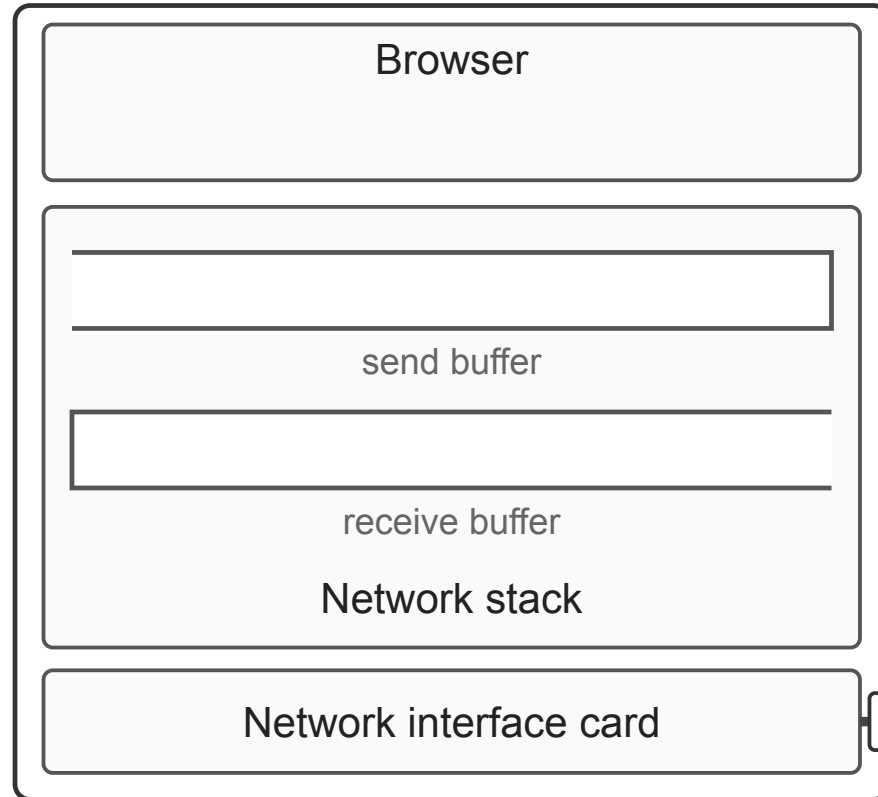


B: server

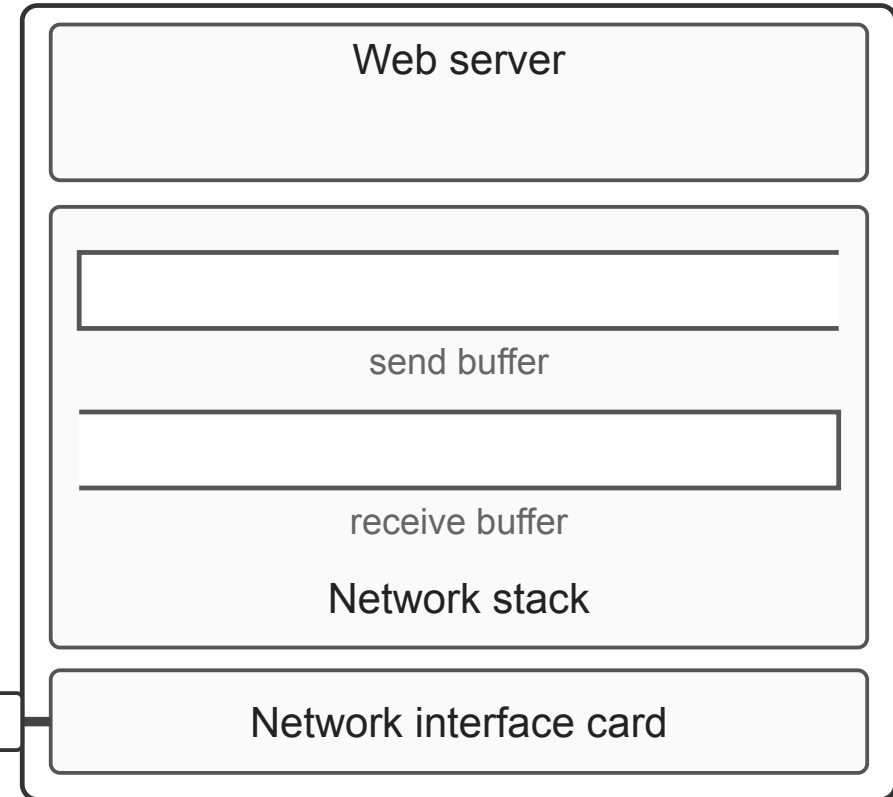


Last time: we built a web browser

A: client



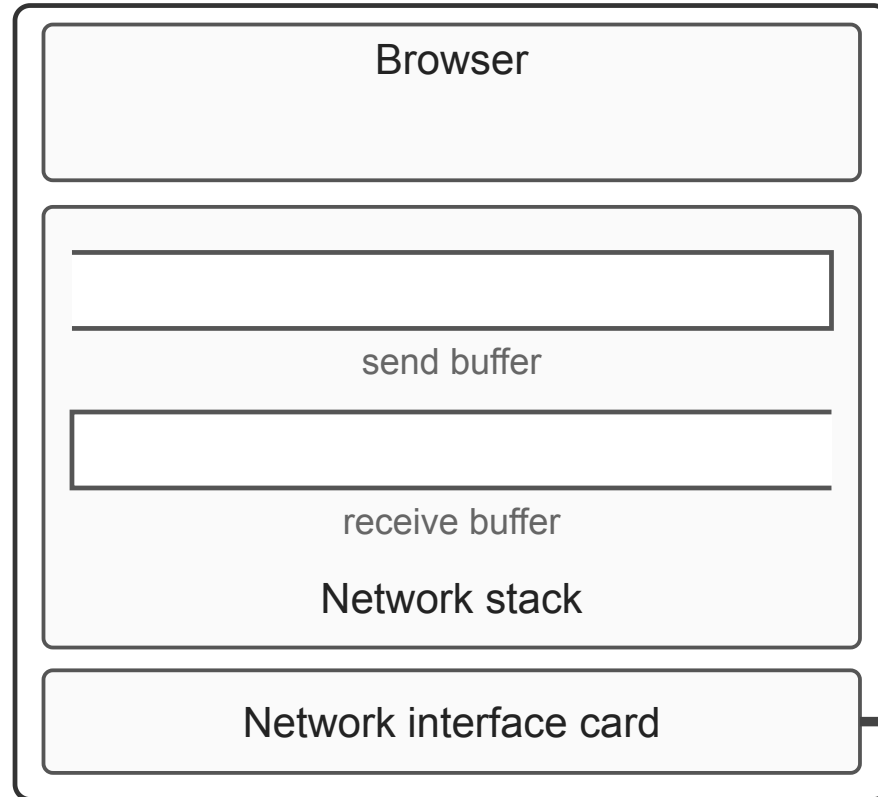
B: server



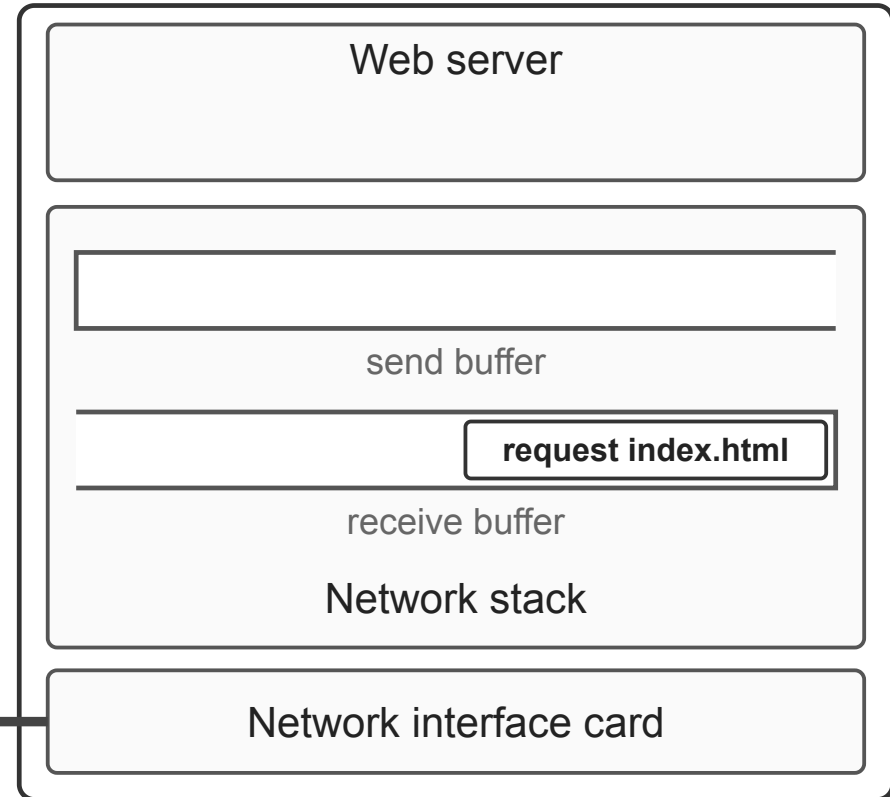
request index.html

Last time: we built a web browser

A: client

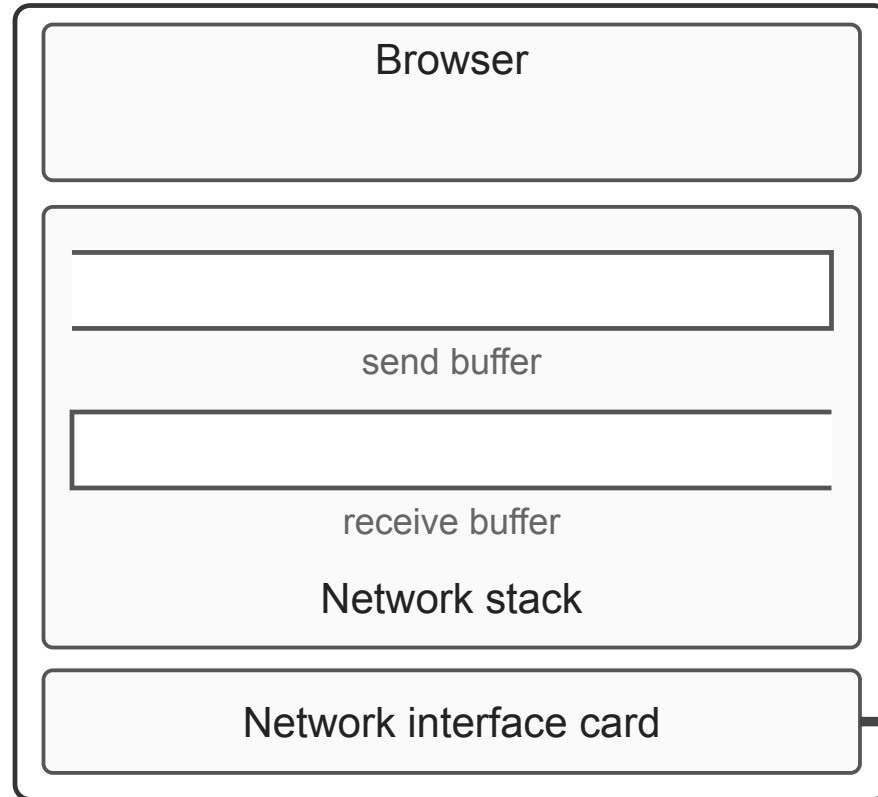


B: server

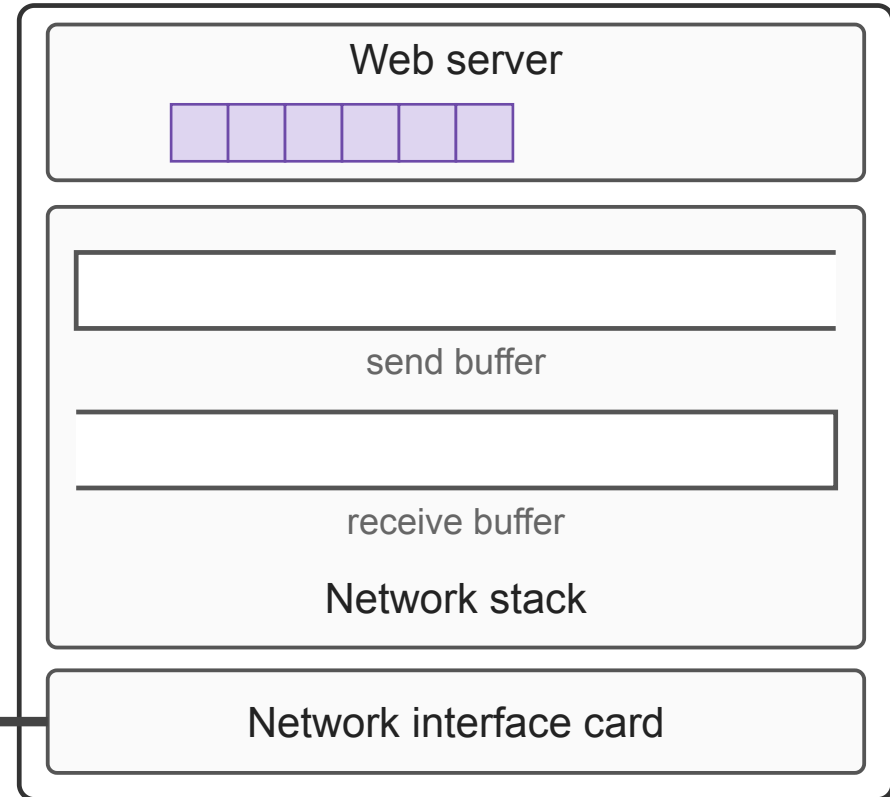


Last time: we built a web browser

A: client

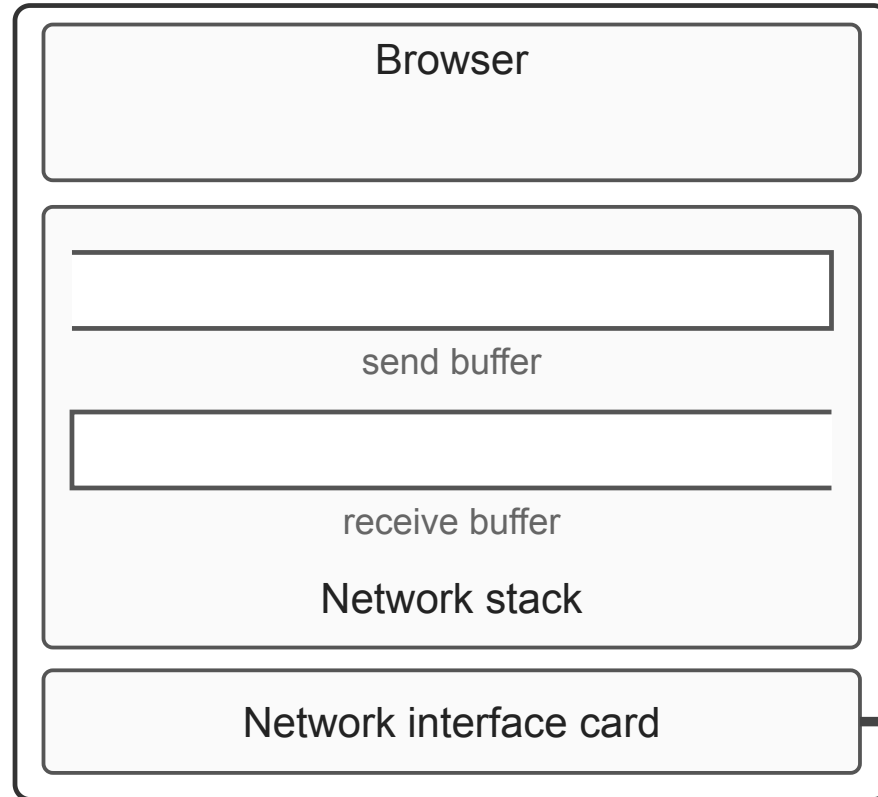


B: server

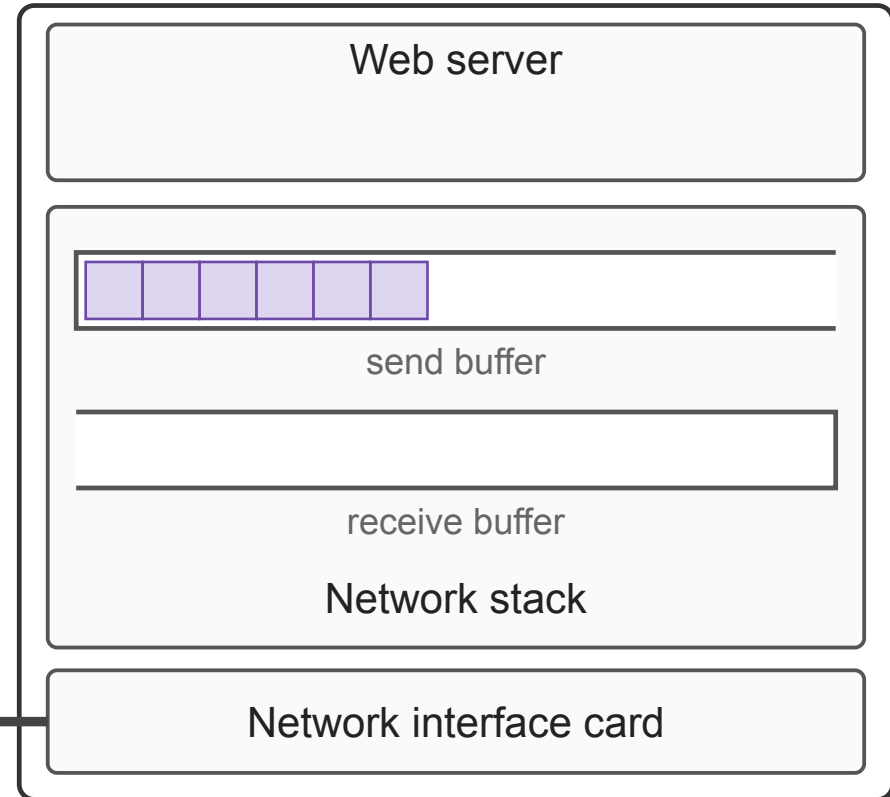


Last time: we built a web browser

A: client

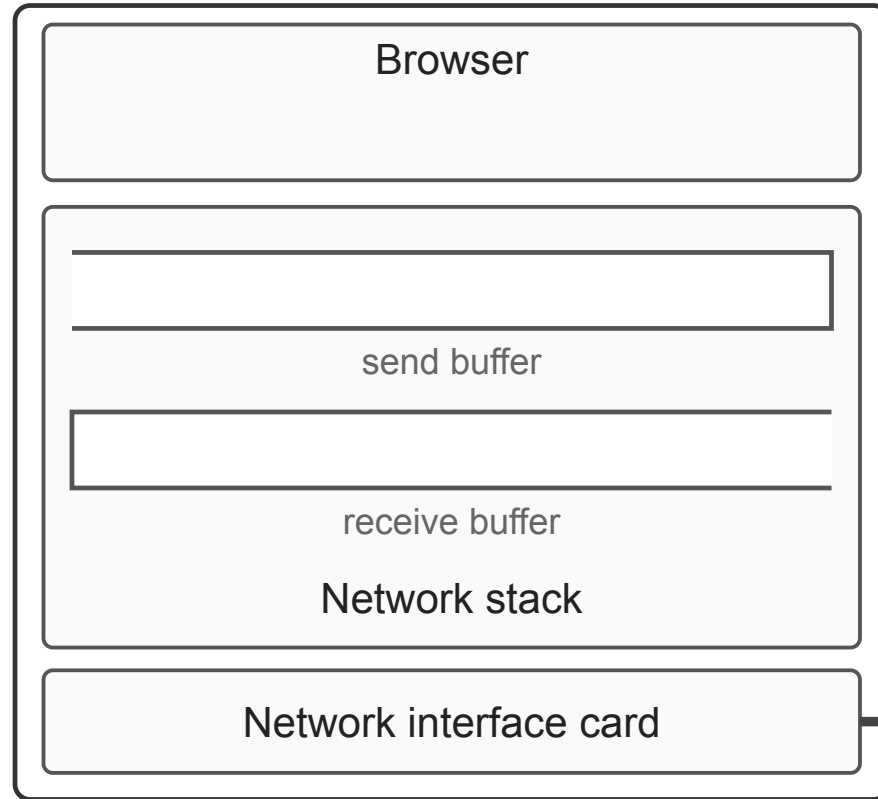


B: server

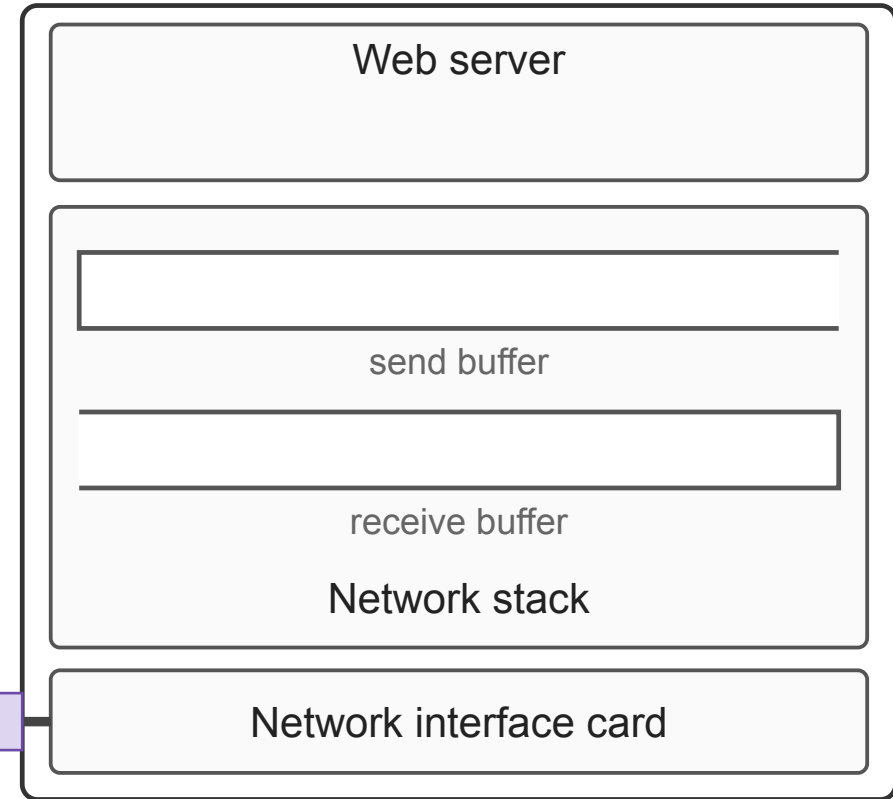


Last time: we built a web browser

A: client

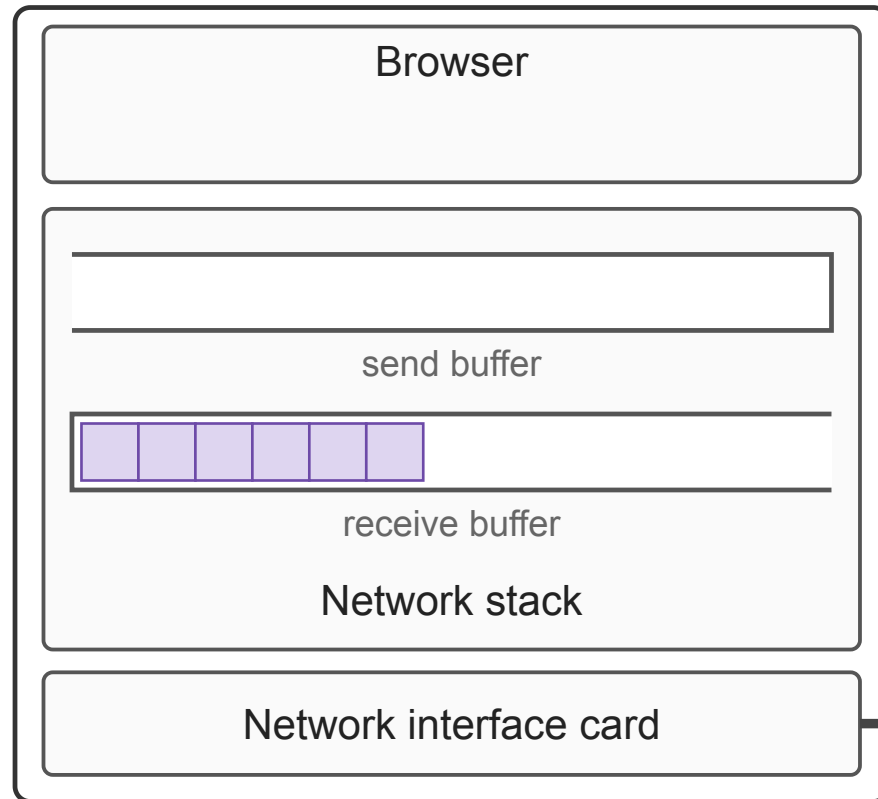


B: server

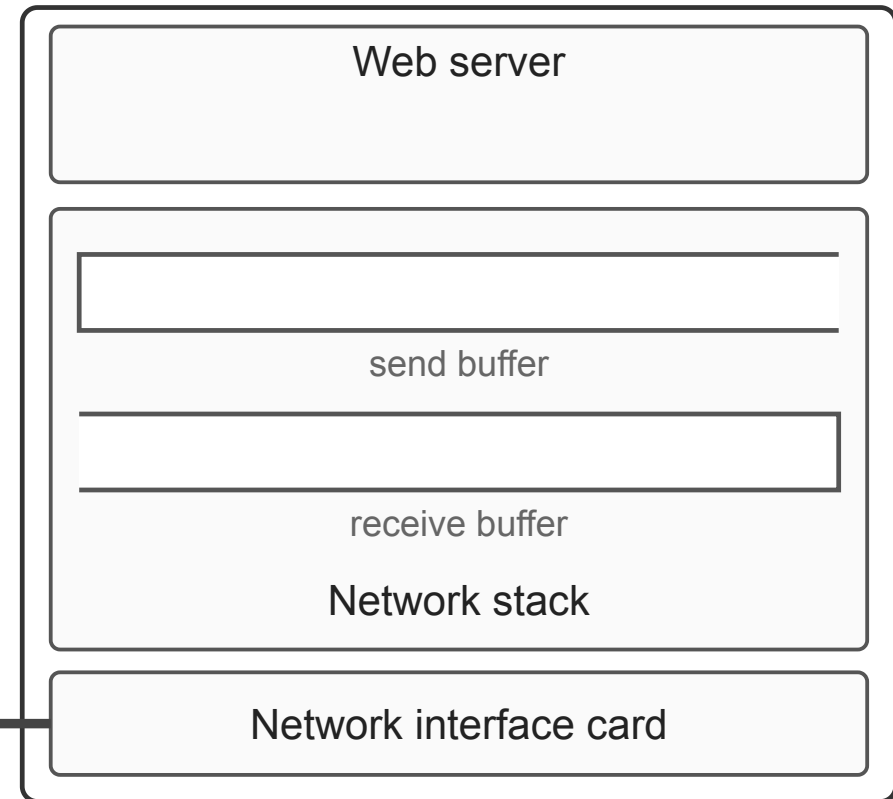


Last time: we built a web browser

A: client

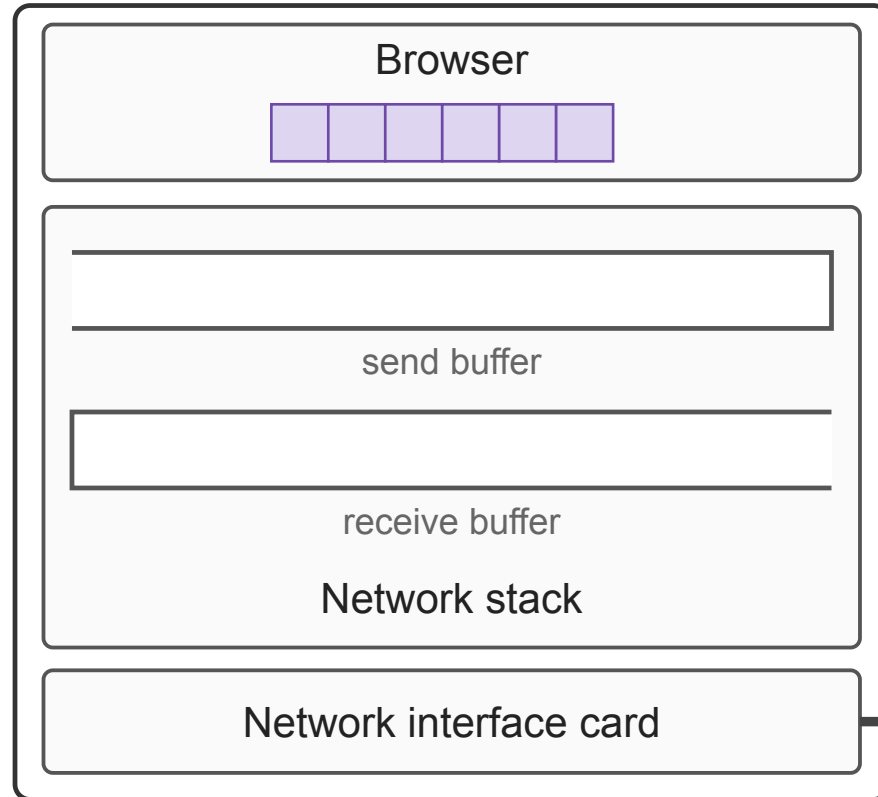


B: server

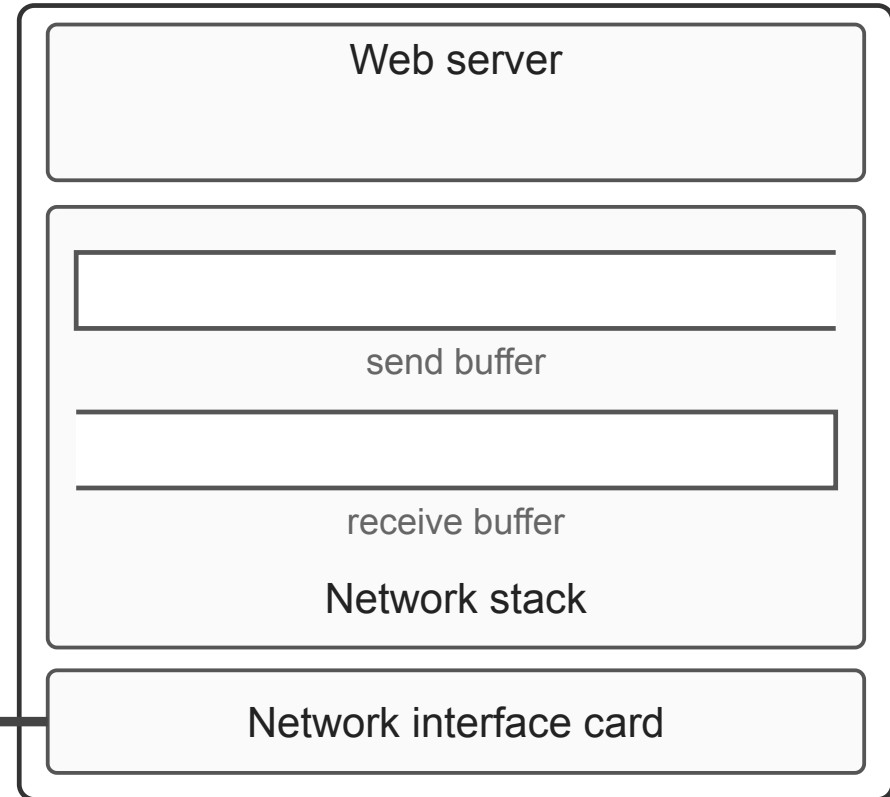


Last time: we built a web browser

A: client



B: server



We also saw

- A page with more than one file, and head-of-line blocking
- Interleaving the files, in the application or in the network stack

Now let's change the application

Outline

- Where we were
- **Video streaming on a perfect network**
- Video streaming on a real network
- What each application wants from the network
- Takeaways

Video streaming

- A video is also a file on a server that the client can ask for
- But nobody wants to wait for the whole file to download before playing the video
 - hence the term streaming
- Only need to show 30 frames every second
 - not the whole file
- The video is cut into chunks of a few seconds
 - the client asks for the next chunk when it needs it

What is one second of video?

30 frames, shown one every $\frac{1}{30}$ s: that is 1 s of video



What is one second of video?

30 frames, shown one every $1/30$ s: that is 1 s of video



the same 30 frames can arrive much faster, e.g., in 0.2 s



What is one second of video?

30 frames, shown one every $1/30$ s: that is 1 s of video



the same 30 frames can arrive much faster, e.g., in 0.2 s



or much slower, e.g., over 3 s



What is one second of video?

30 frames, shown one every $1/30$ s: that is 1 s of video



the same 30 frames can arrive much faster, e.g., in 0.2 s



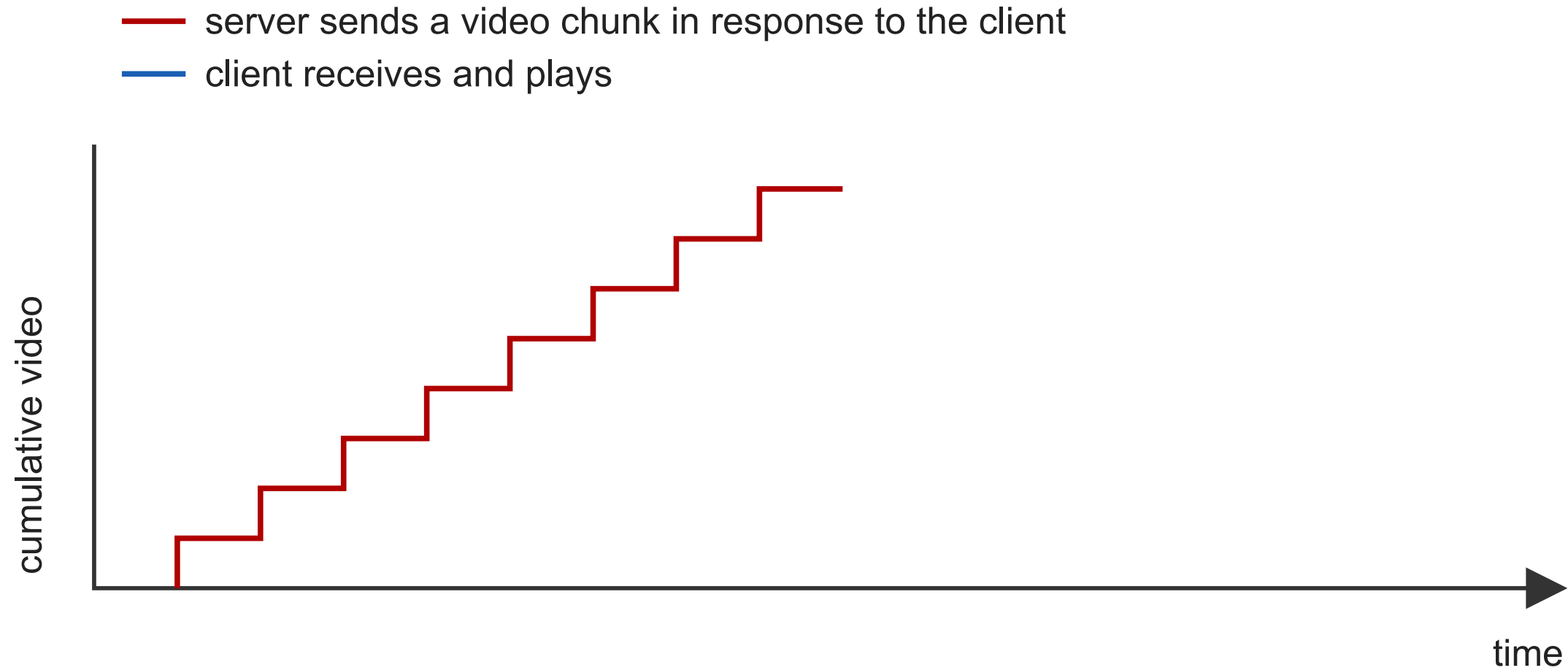
or much slower, e.g., over 3 s



Seconds of video: how long it plays, not how long it takes to arrive

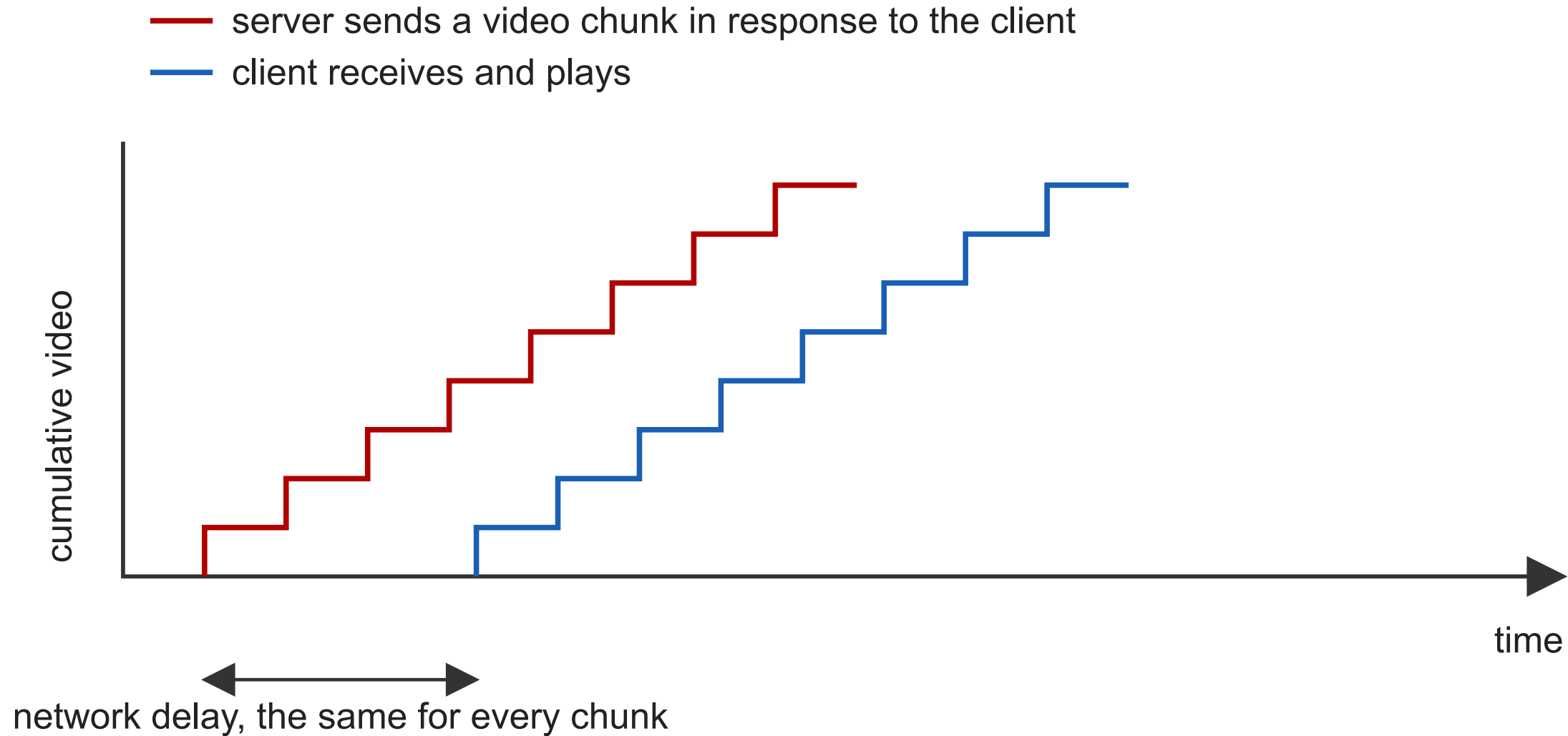
On a perfect network

these graphs simplify what really happens



On a perfect network

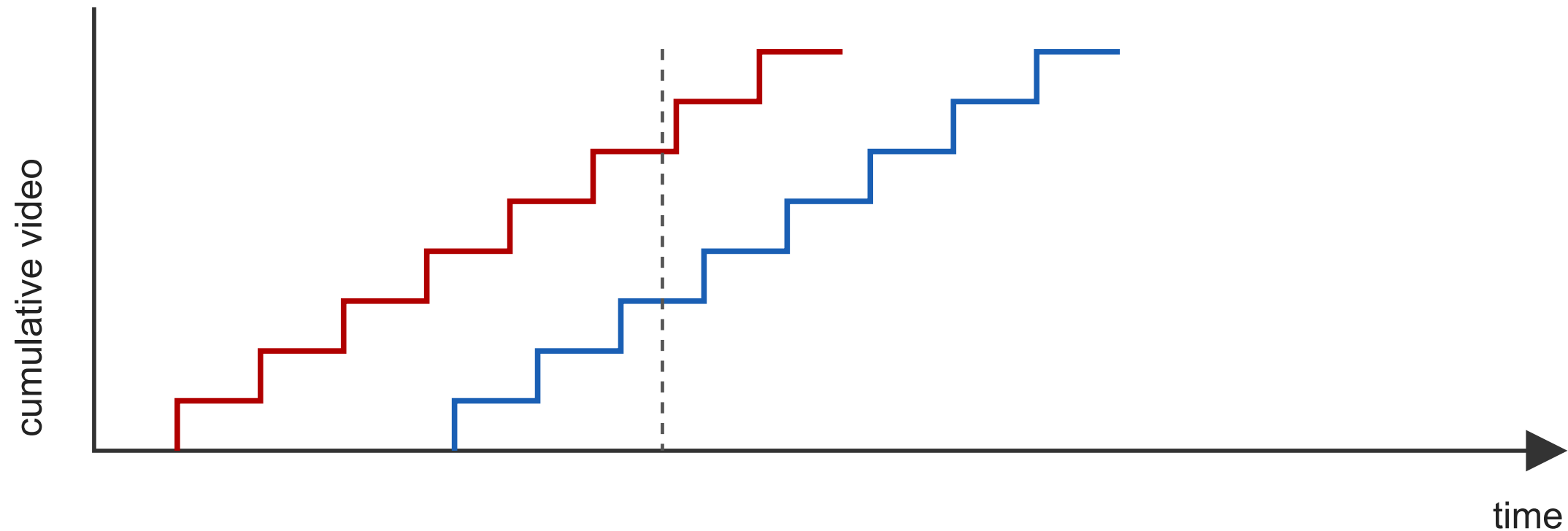
these graphs simplify what really happens



On a perfect network

these graphs simplify what really happens

- server sends a video chunk in response to the client
- client receives and plays

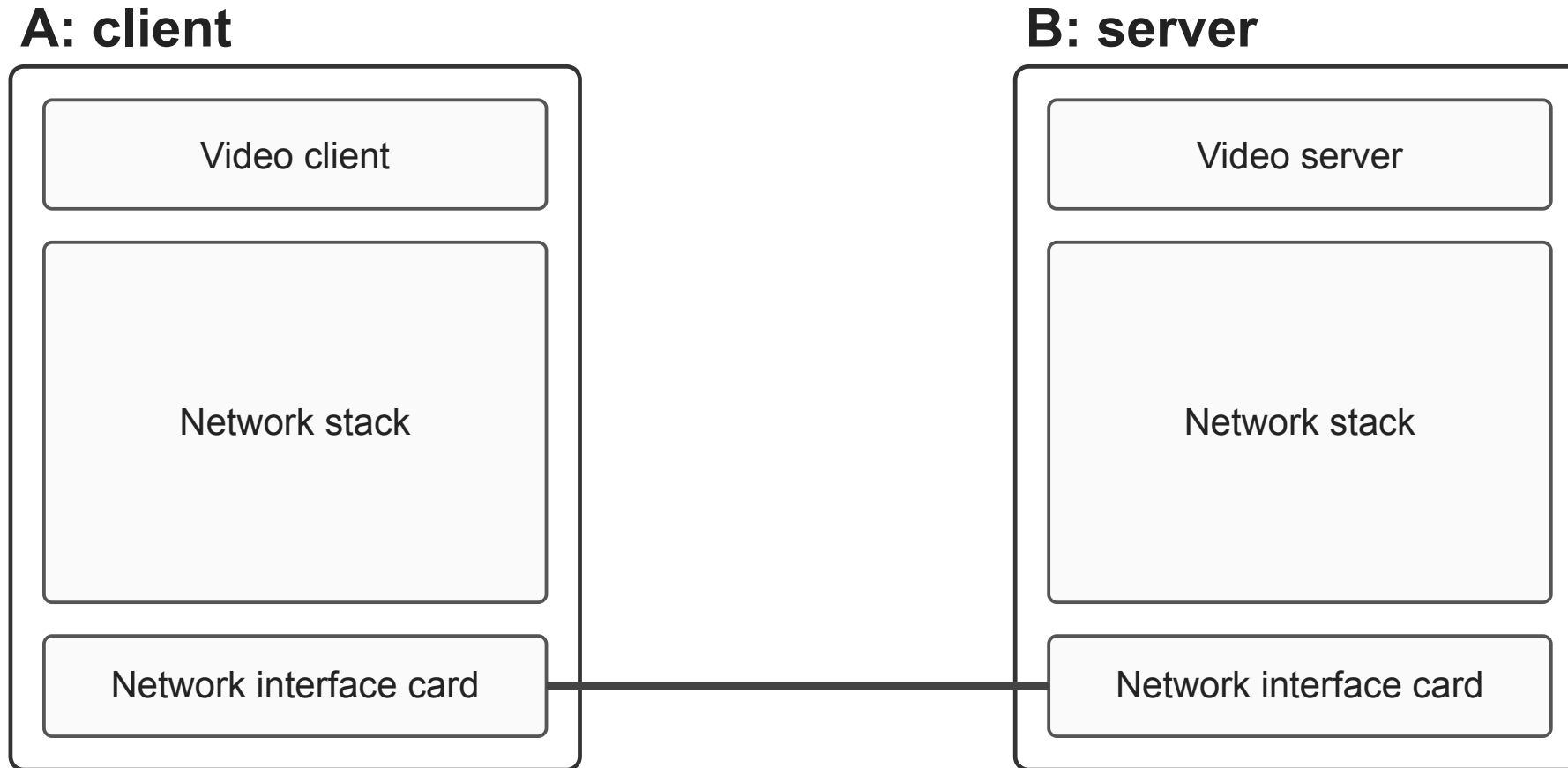


network delay, the same for every chunk

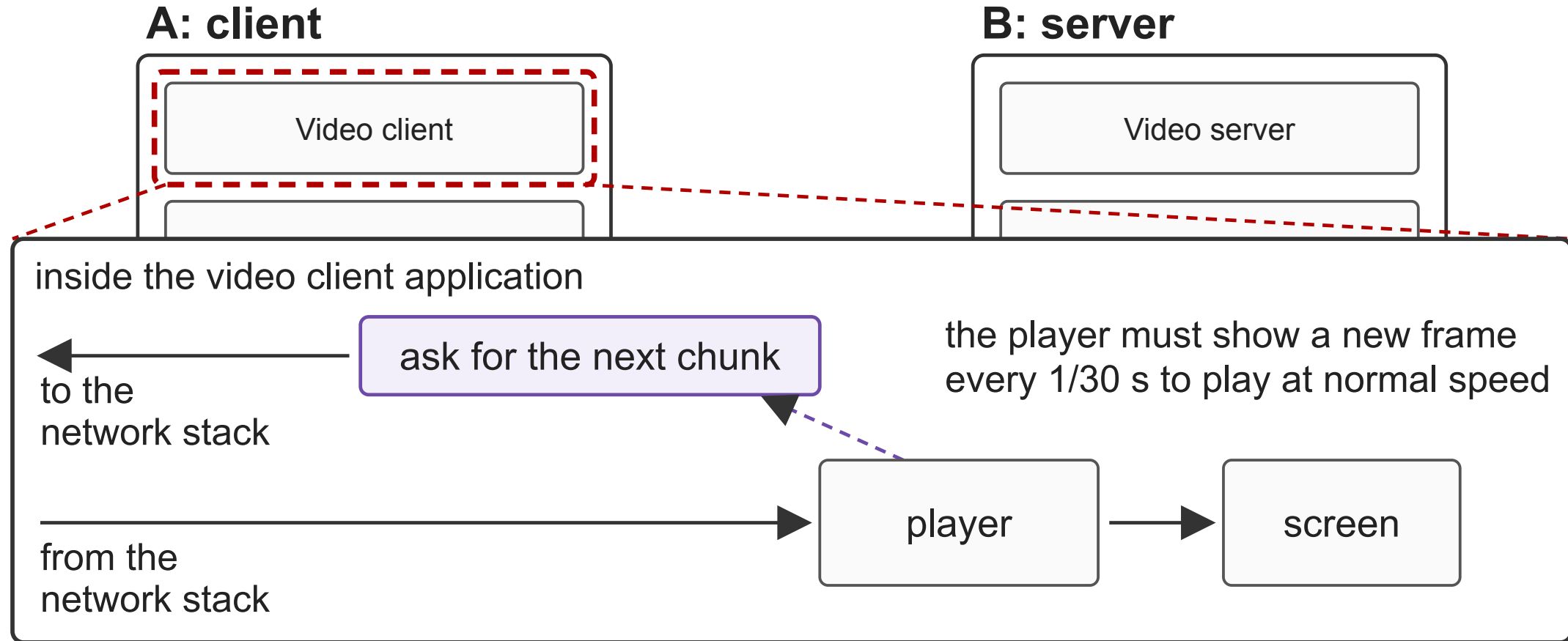
right now: the client plays early chunks while it is still asking for later ones

How would you build the client?

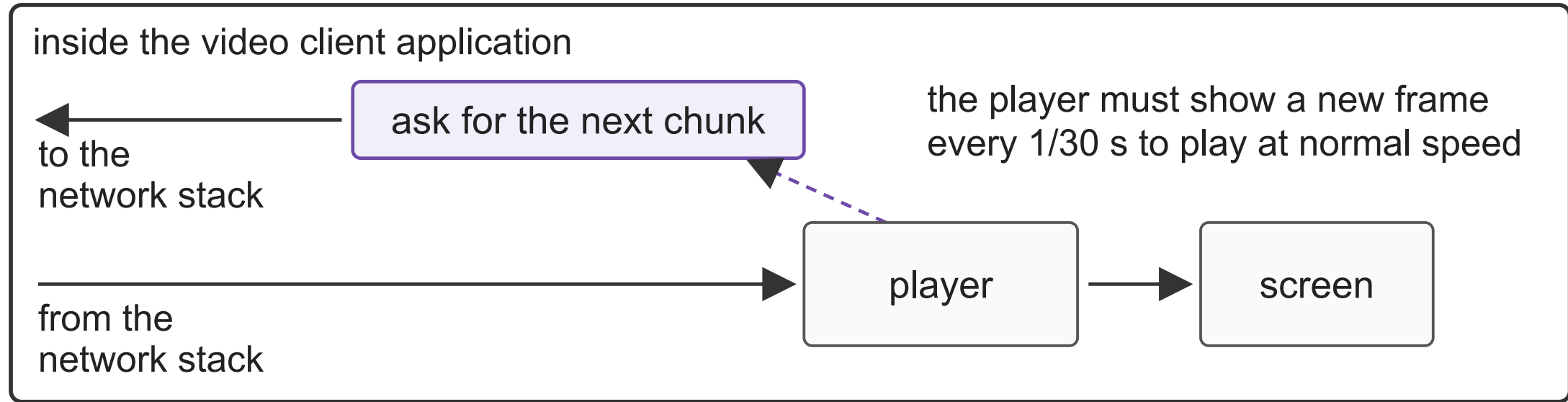
Inside the video client



Inside the video client



On a perfect network: ask one delay ahead



ask for each chunk one network delay before the player needs it
every $1/30$ s: show the next frame

- The delay is the same for every chunk, so the client knows exactly how far ahead to ask

Outline

- Where we were
- Video streaming on a perfect network
- **Video streaming on a real network**
- What each application wants from the network
- Takeaways

A real network is not a perfect pipe

- The delay may change from one video chunk to the next

A real network is not a perfect pipe

- The delay may change from one video chunk to the next

How much the delay changes is called jitter

Why does the delay change?

$$\text{delay} = 2 \times (d_{\text{proc,A}} + d_{\text{prop}} + d_{\text{proc,B}}) + L_{\text{request}} / R + L_{\text{file}} / R$$

Why does the delay change?

$$\text{delay} = 2 \times (d_{\text{proc,A}} + d_{\text{prop}} + d_{\text{proc,B}}) + L_{\text{request}} / R + L_{\text{file}} / R$$

$$\text{delay} = \text{processing} + \text{propagation} + \text{transmission}$$

Why does the delay change?

$$\text{delay} = 2 \times (d_{\text{proc,A}} + d_{\text{prop}} + d_{\text{proc,B}}) + L_{\text{request}} / R + L_{\text{file}} / R$$

delay = processing + propagation + transmission + waiting

Waiting delay: the time data waits in line because other data got there first and is still going into the pipe. We come back to this in Part 2.

Why does the delay change?

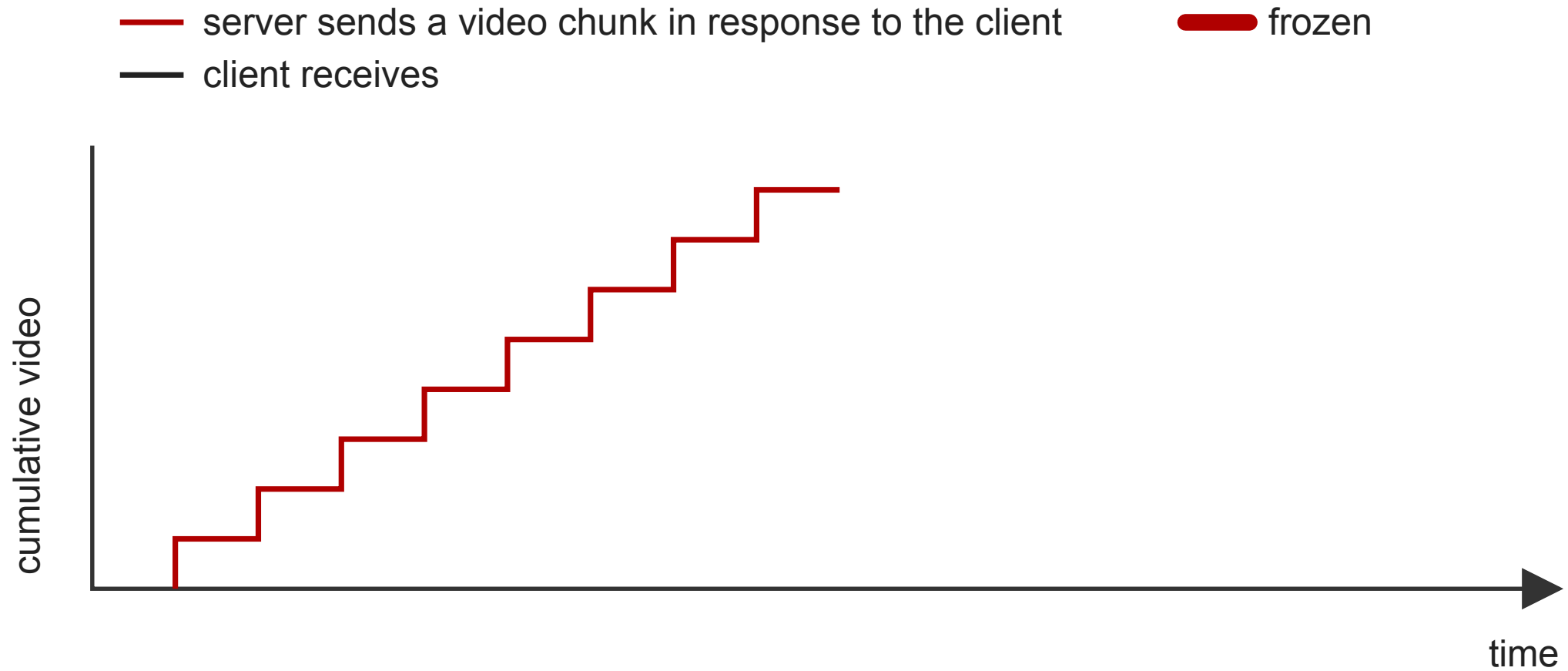
$$\text{delay} = 2 \times (d_{\text{proc,A}} + d_{\text{prop}} + d_{\text{proc,B}}) + L_{\text{request}} / R + L_{\text{file}} / R$$

delay = processing + propagation + transmission + waiting

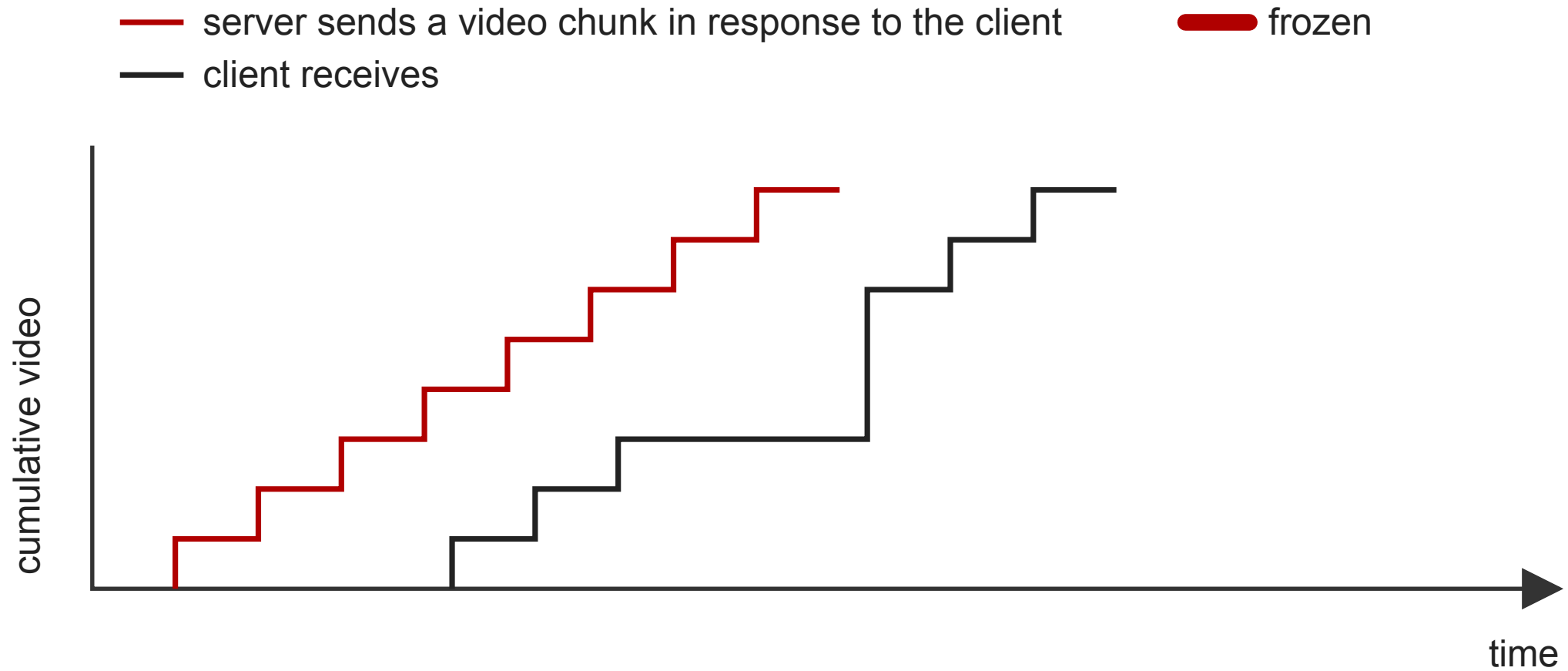
Waiting delay: the time data waits in line because other data got there first and is still going into the pipe. We come back to this in Part 2.

Each of these can change, sometimes temporarily
or for longer stretches of time throughout the
streaming session

A chunk arrives late

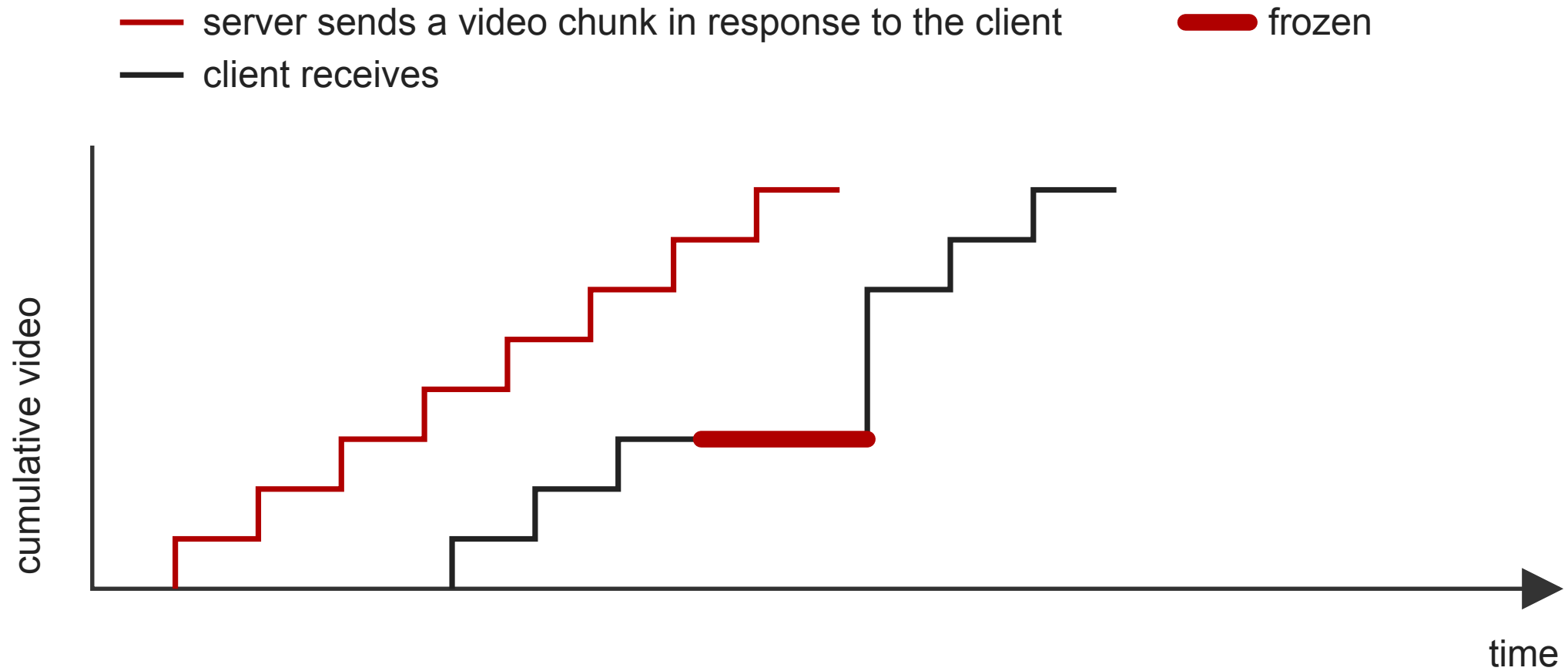


A chunk arrives late



for a short while the delay goes up, then it goes back to normal

A chunk arrives late



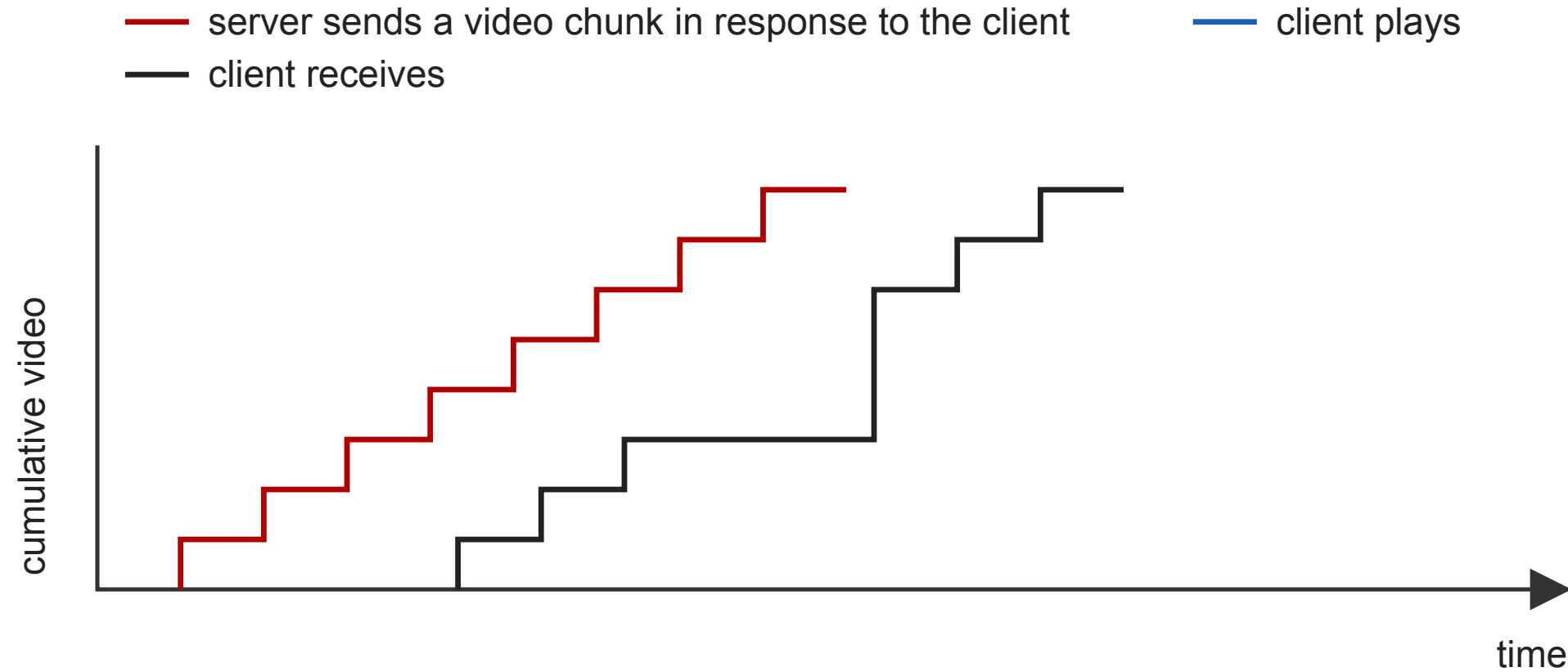
for a short while the delay goes up, then it goes back to normal

if the client starts playing the moment the chunks arrive,

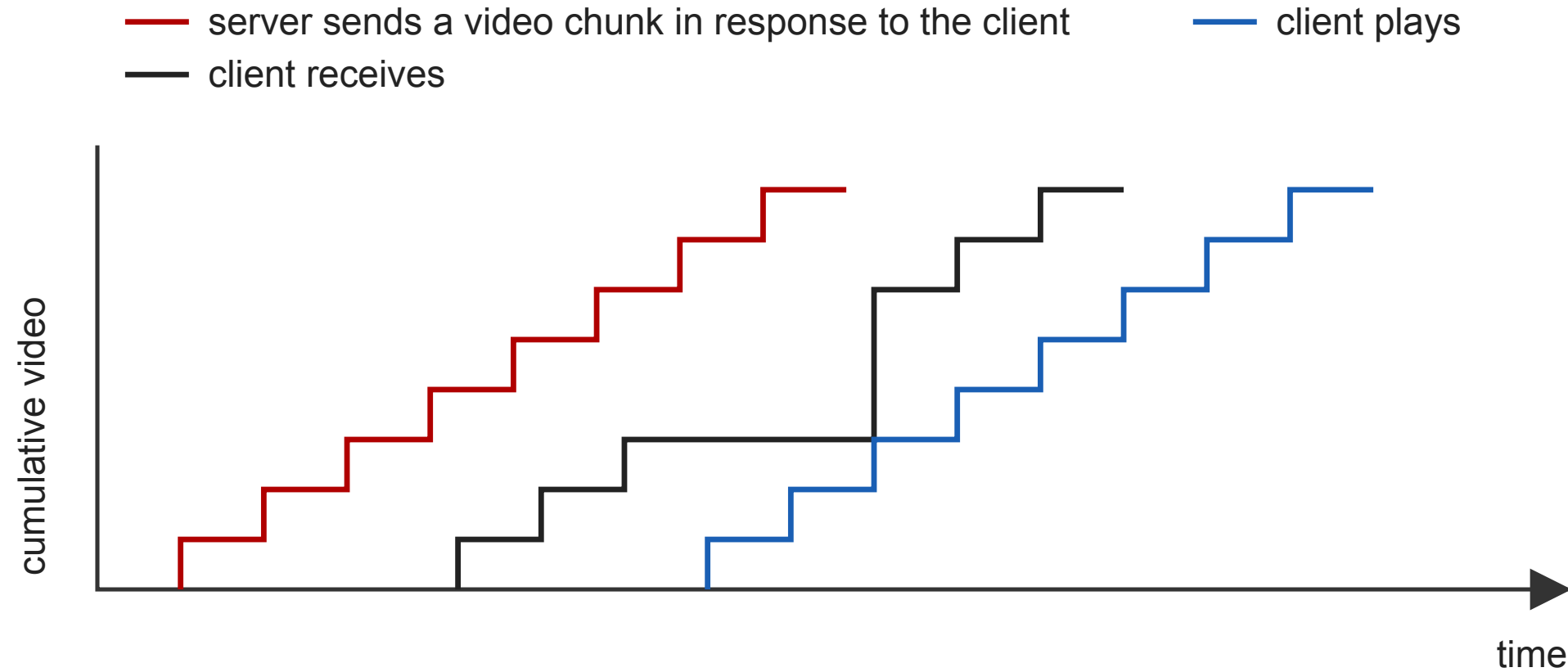
when chunks are delayed, the video freezes until the late chunks show up

How would you change the client now?

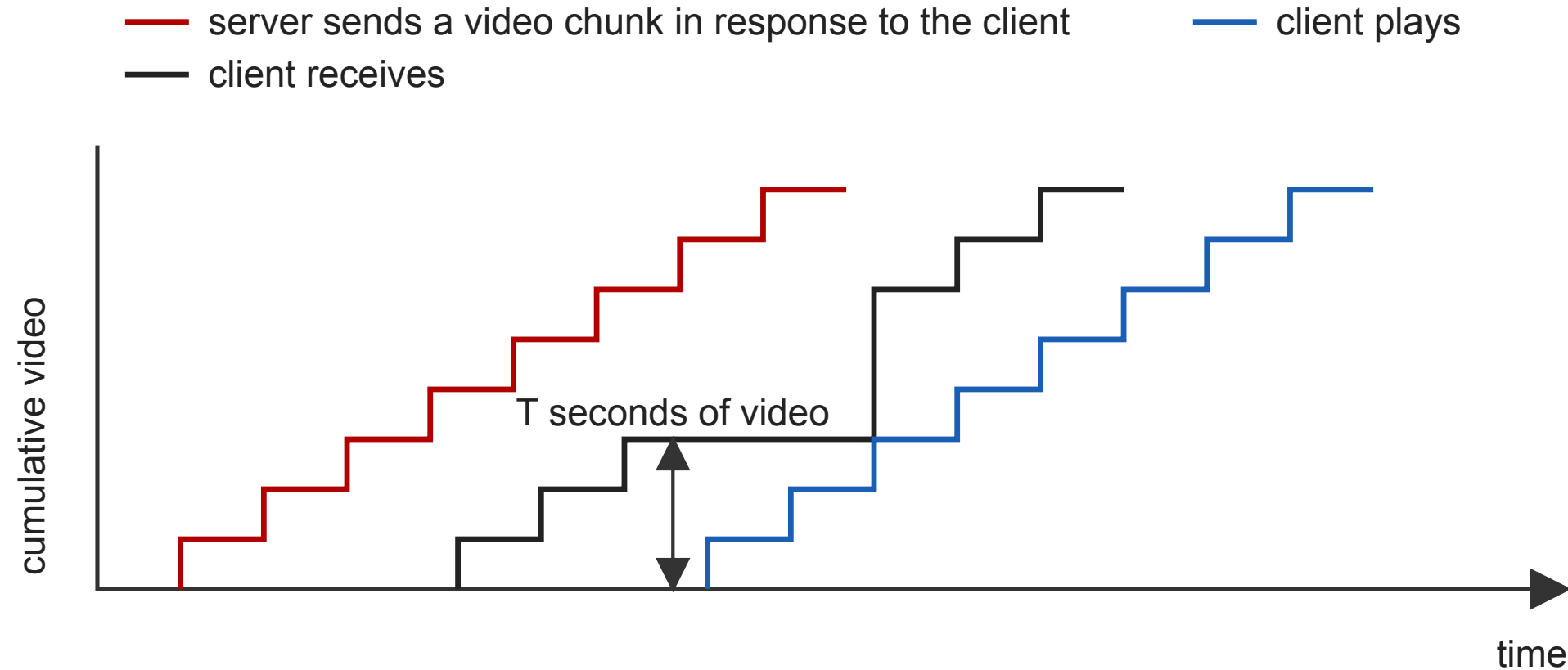
Wait before playing



Wait before playing

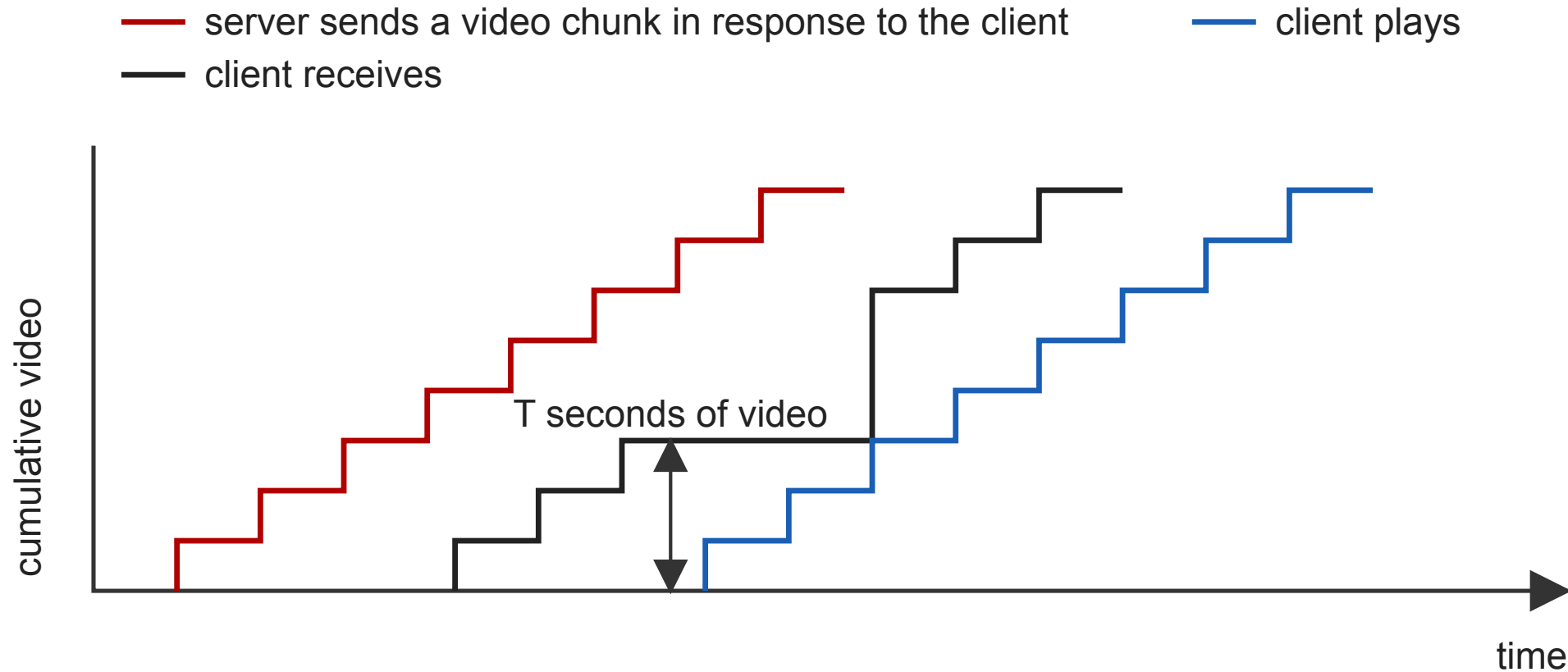


Wait before playing



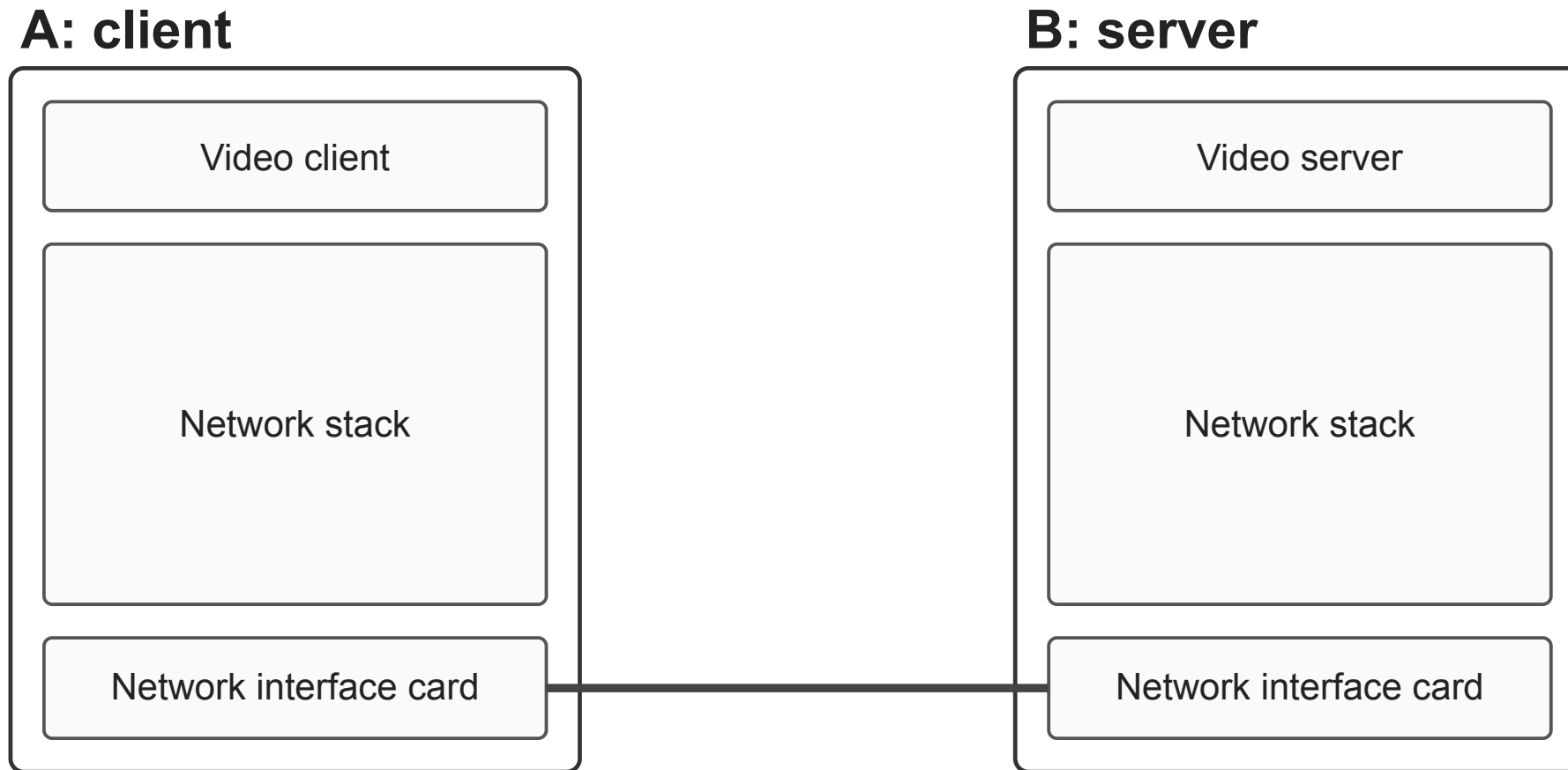
wait until the playback buffer holds T seconds of video, then start playing

Wait before playing

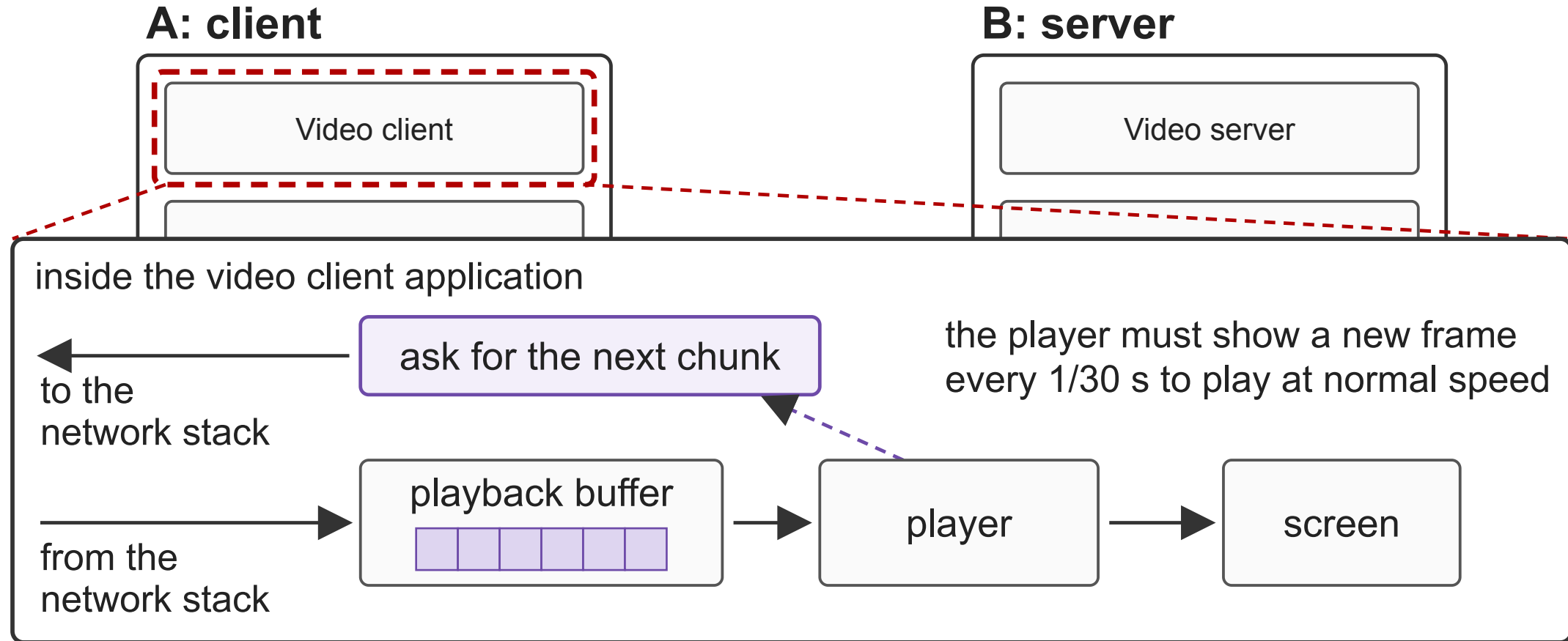


wait until the playback buffer holds T seconds of video, then start playing
if a chunk is delayed, the player can use the buffered video
while the late chunks are on their way

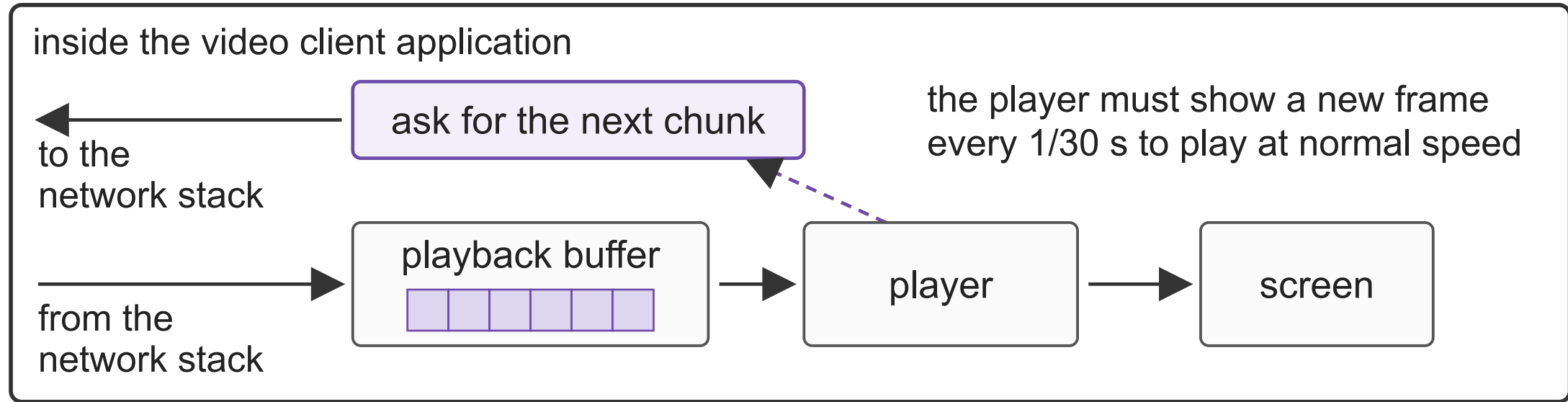
Back inside the client



Back inside the client



Wait before playing



buffer just above T seconds: ask for the next chunk

every $1/30$ s: show the next frame, or freeze if there is none

- T is how far ahead the client asks, in seconds of video
 - bigger T : fewer freezes, but a later start

What if the change lasts a while?

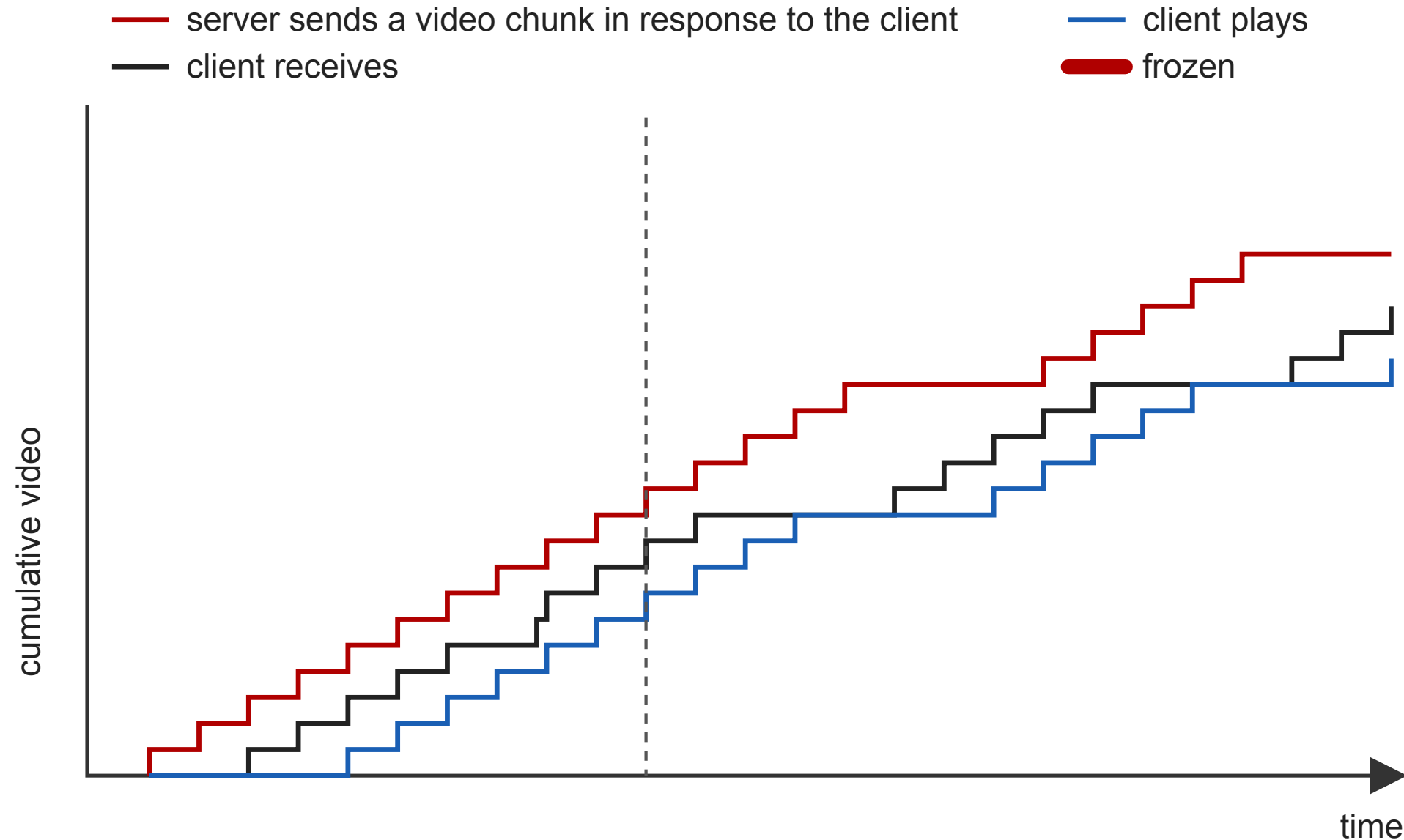
- So far, the delay went up and down only for a few chunks, so the playback buffer could smooth it out
- Sometimes it changes and stays that way for a while:
 - you walk from Wi-Fi to cellular
 - more people start using the network
 - your traffic starts taking a different path, longer or shorter, with a different rate

What if the change lasts a while?

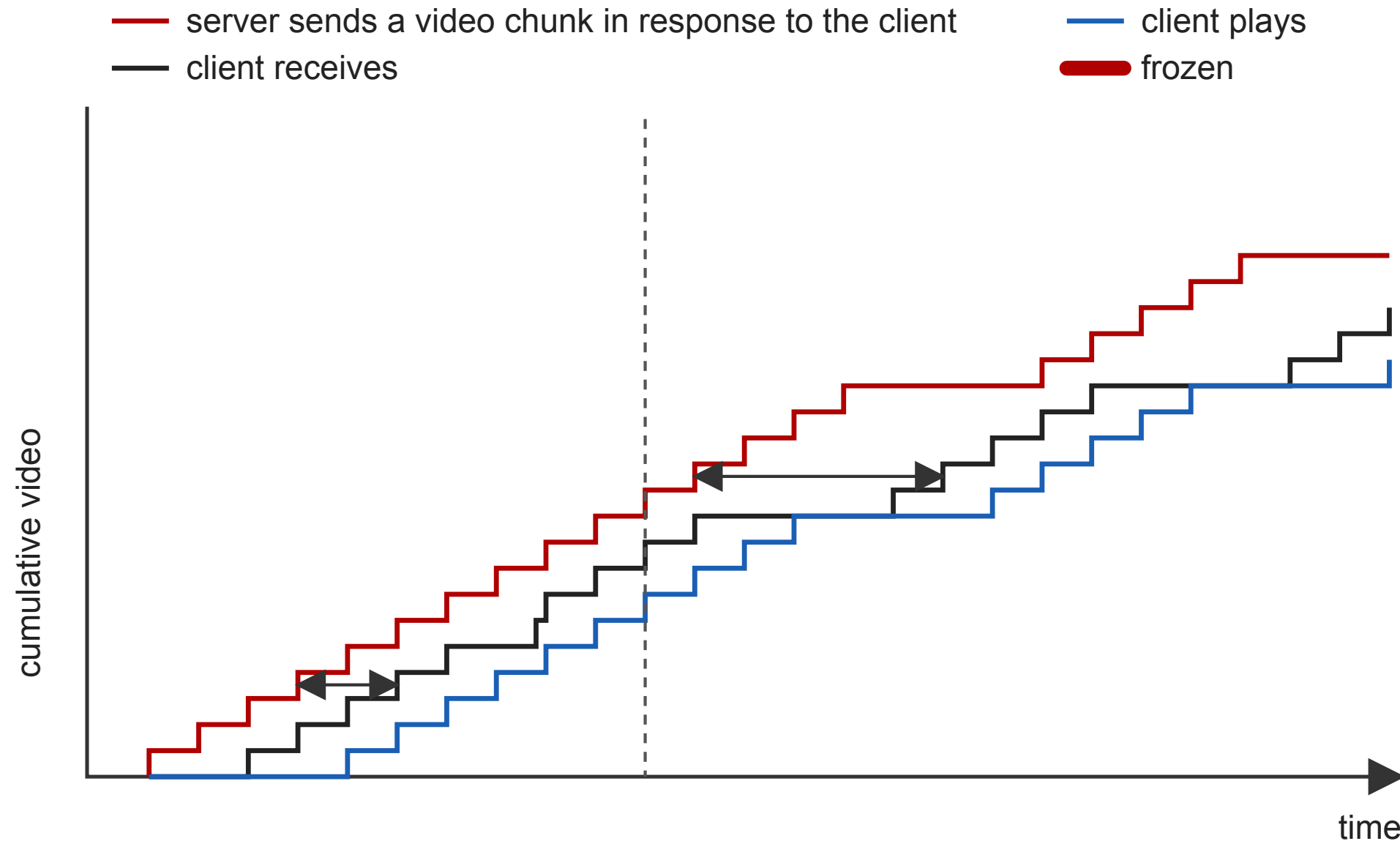
- So far, the delay went up and down only for a few chunks, so the playback buffer could smooth it out
- Sometimes it changes and stays that way for a while:
 - you walk from Wi-Fi to cellular
 - more people start using the network
 - your traffic starts taking a different path, longer or shorter, with a different rate

Which factors that impact the delay can each of these change?

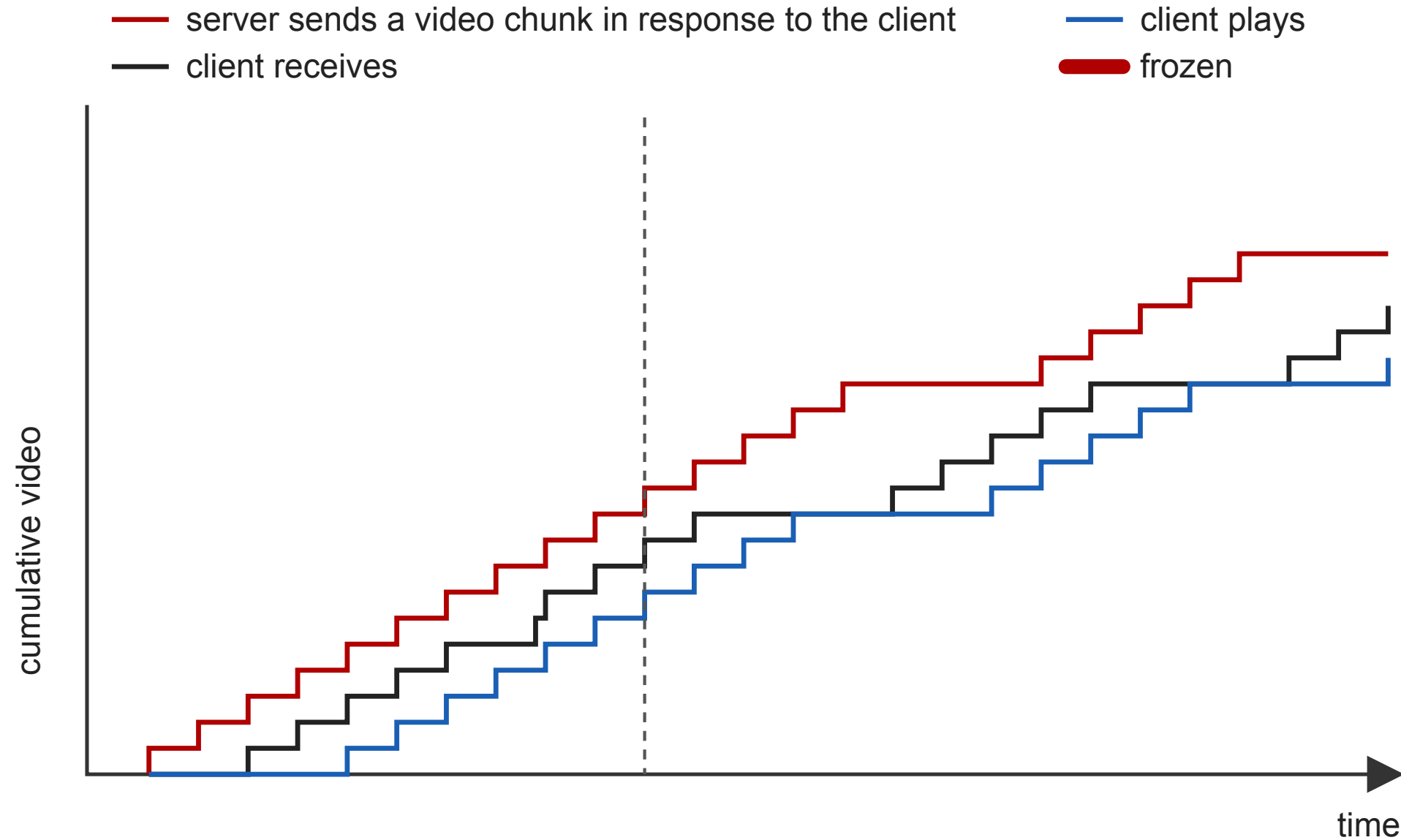
If the pipe gets longer for a while



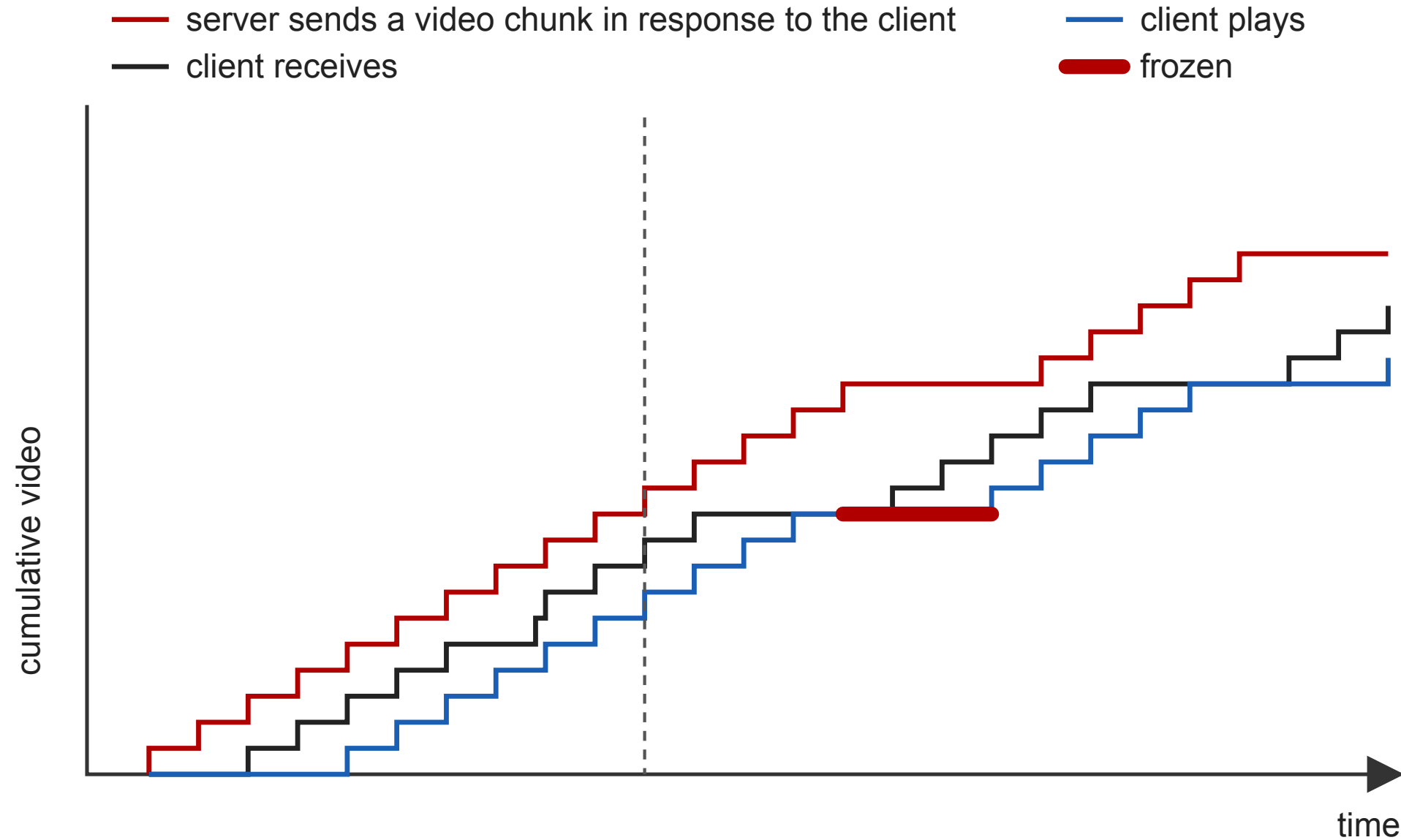
If the pipe gets longer for a while



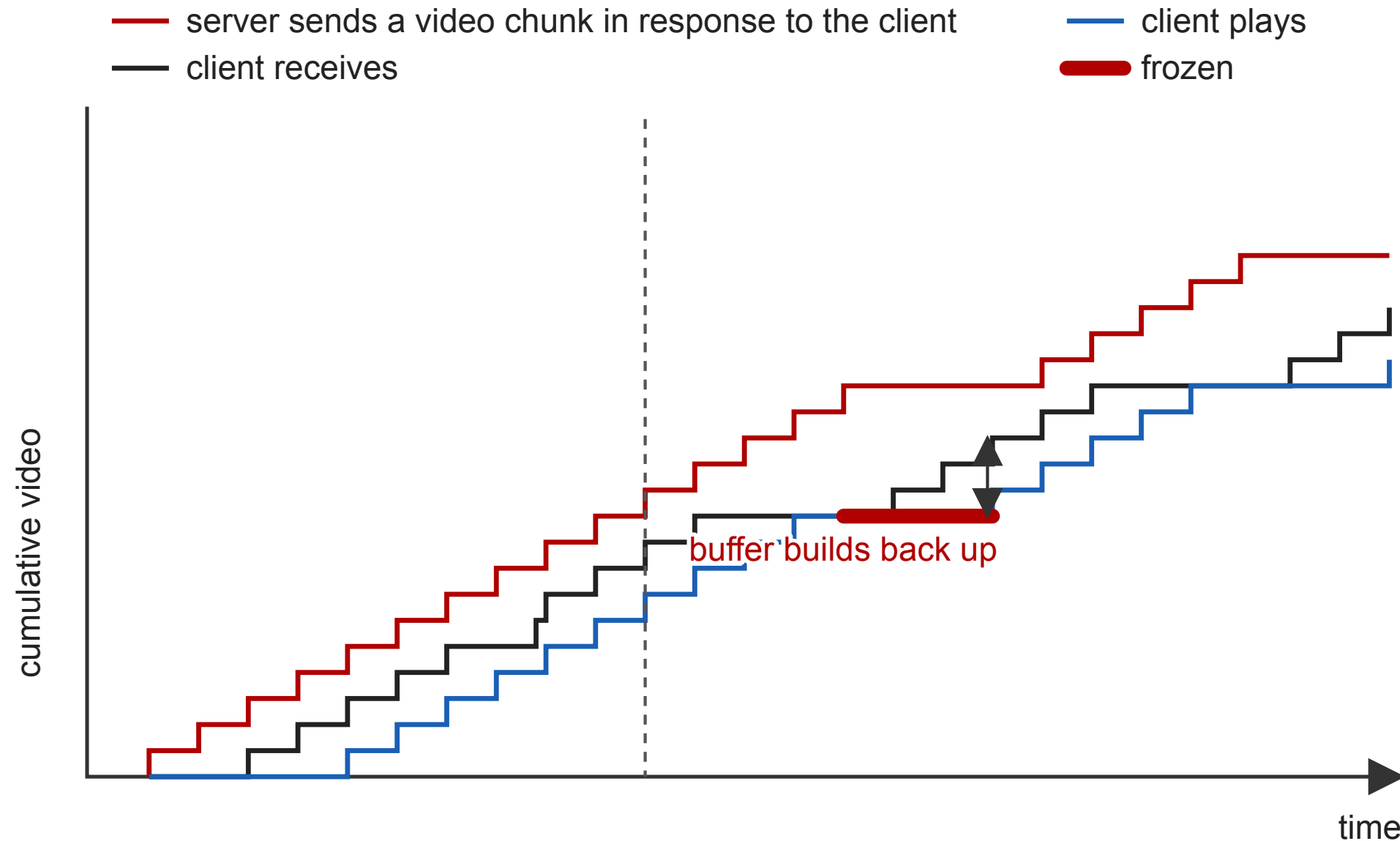
If the pipe gets longer for a while



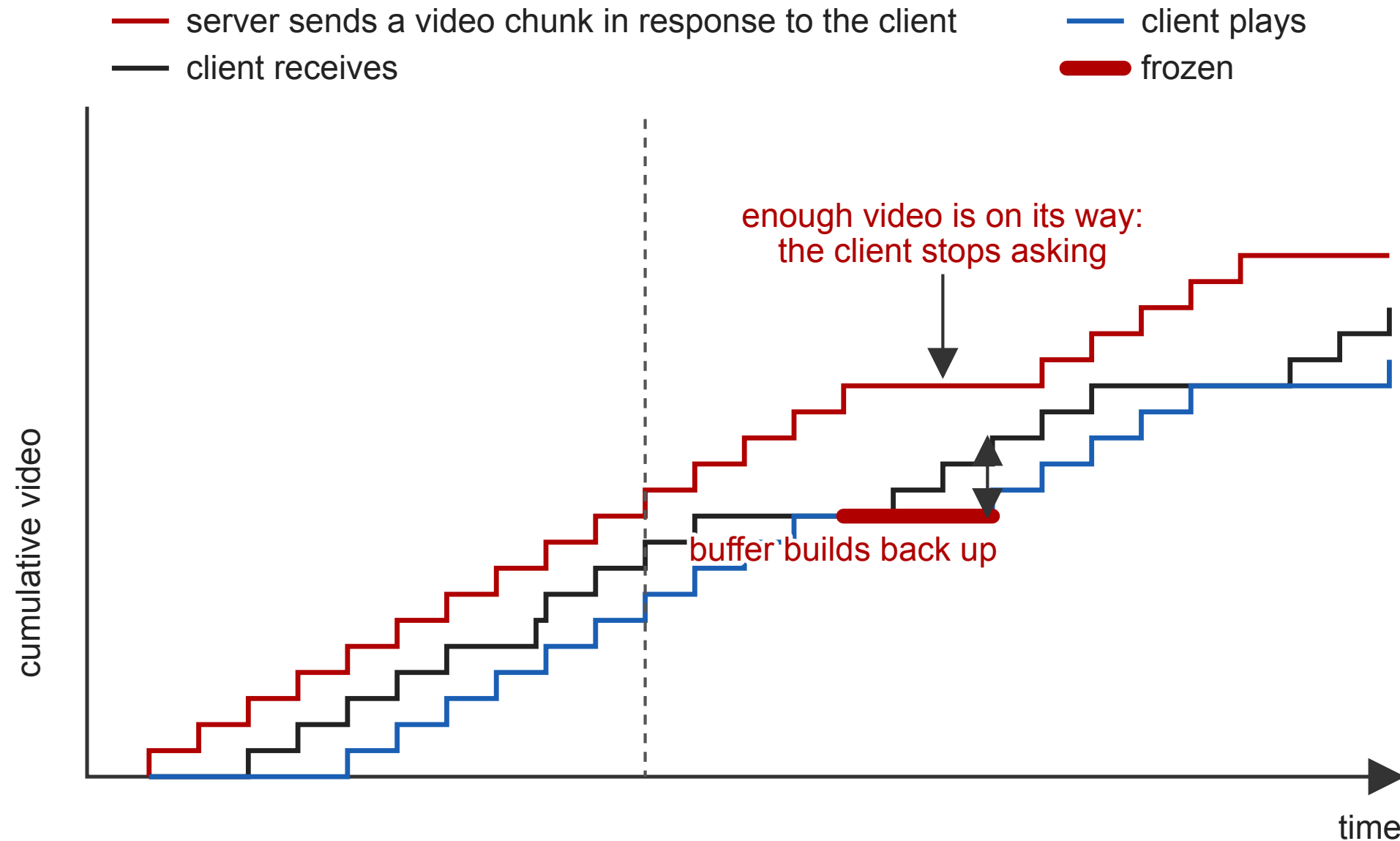
If the pipe gets longer for a while



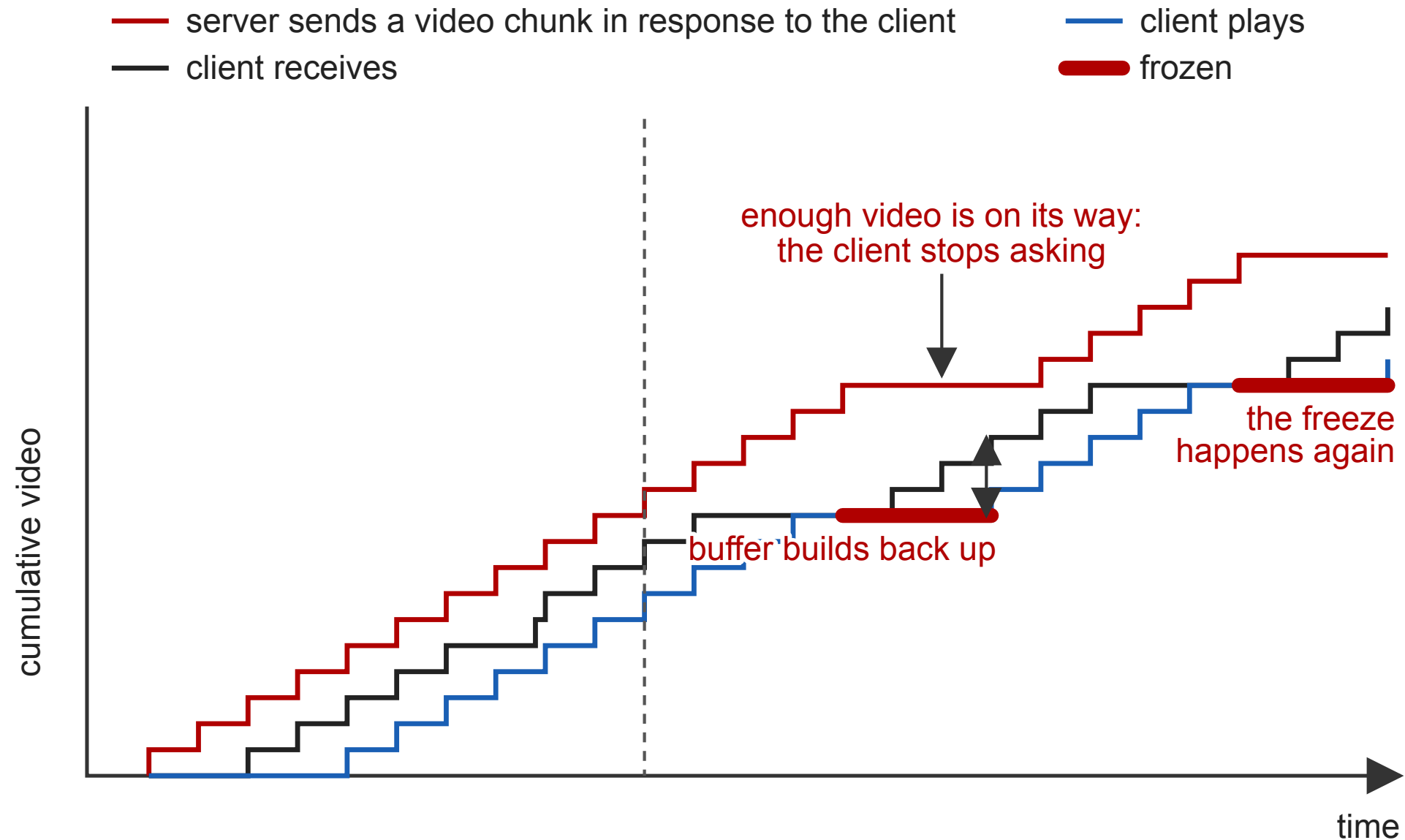
If the pipe gets longer for a while



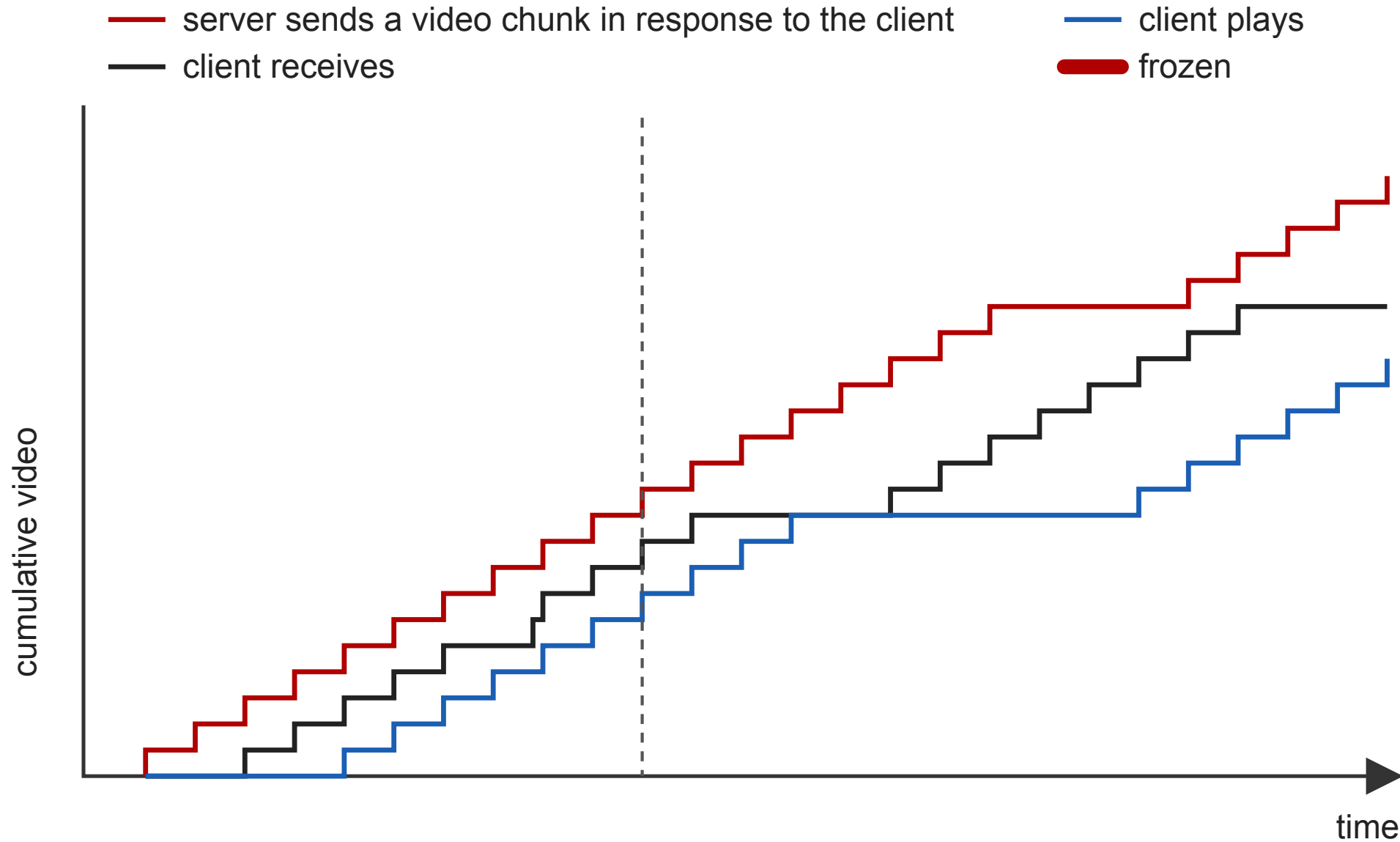
If the pipe gets longer for a while



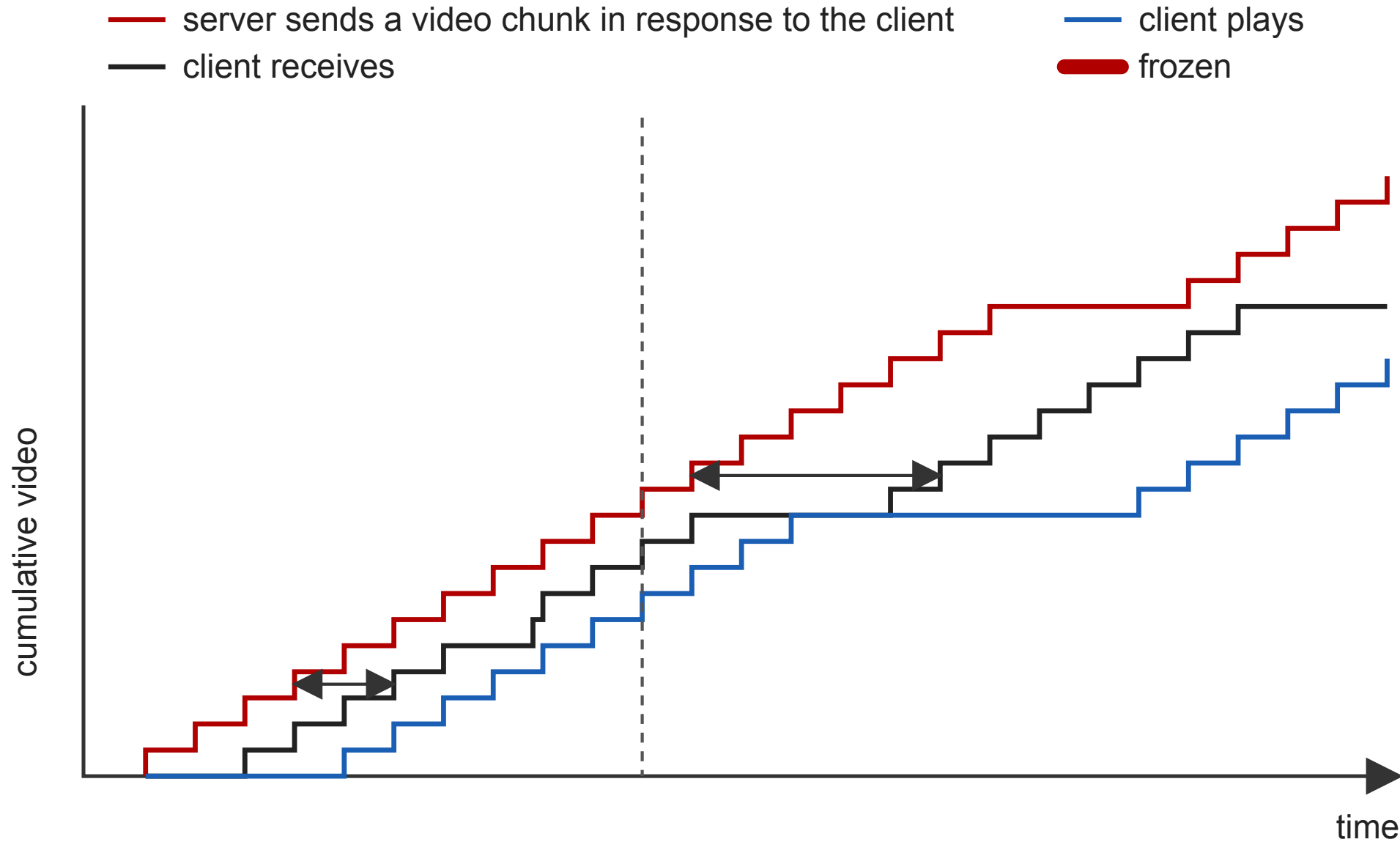
If the pipe gets longer for a while



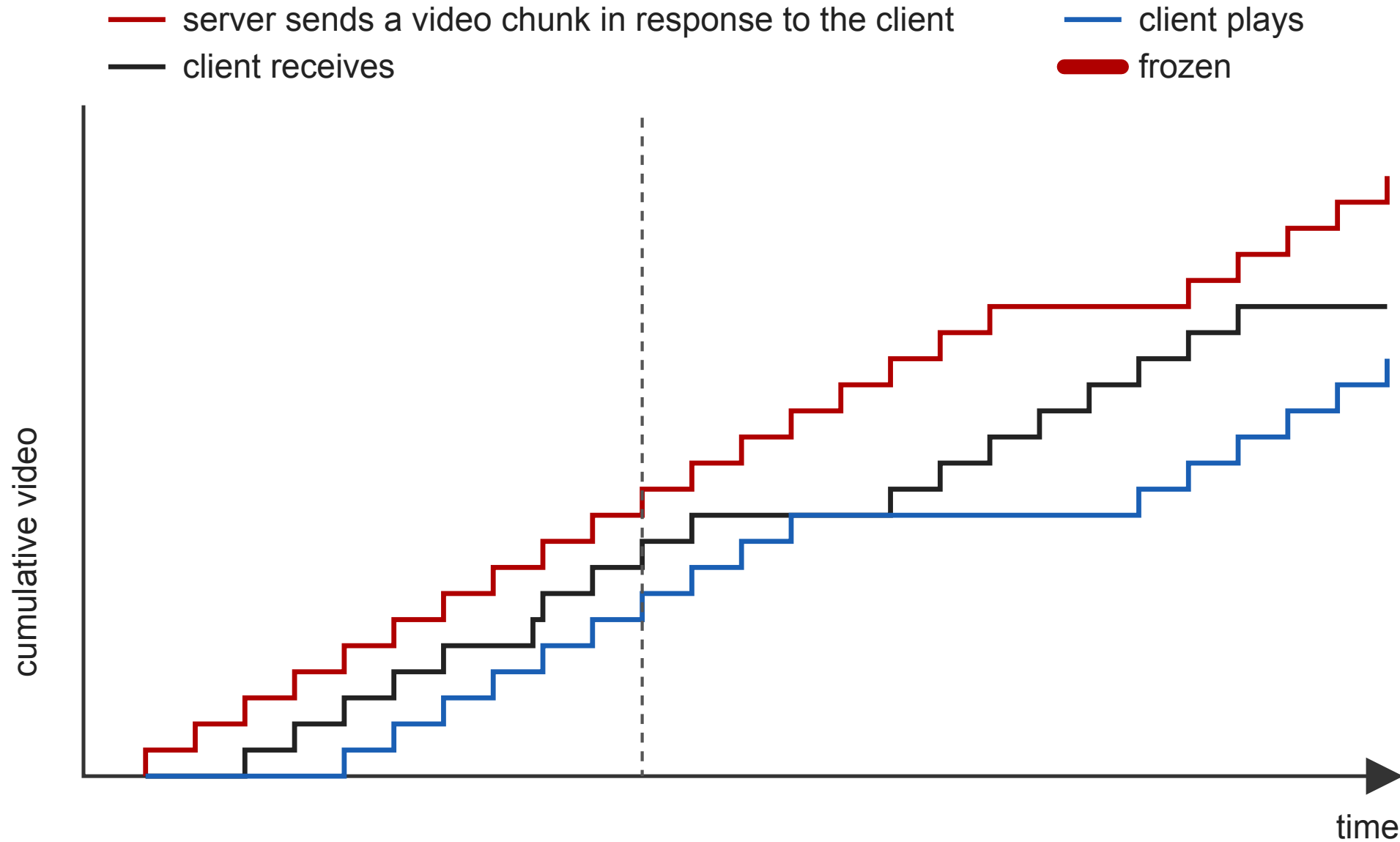
A longer pipe: use a bigger T



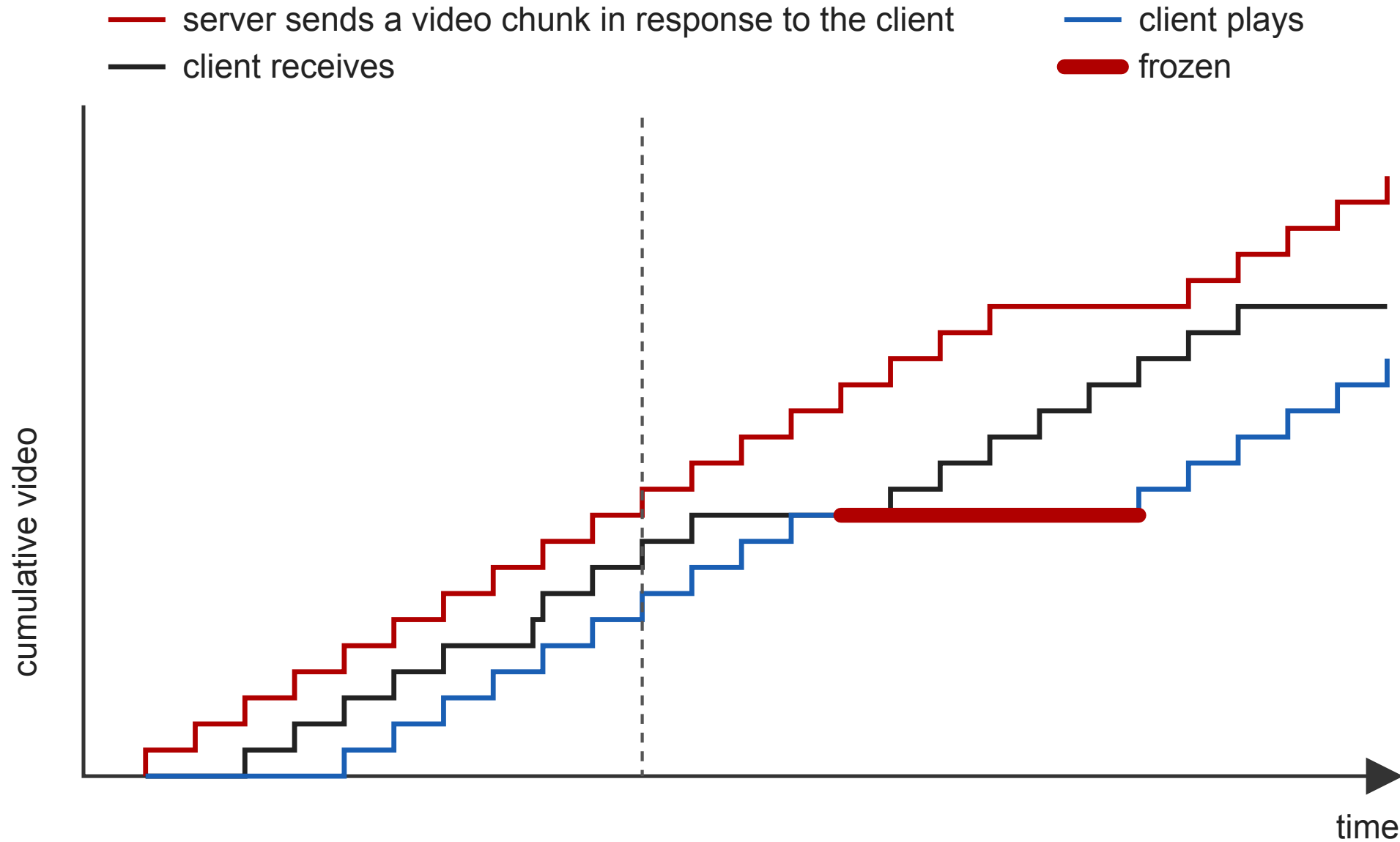
A longer pipe: use a bigger T



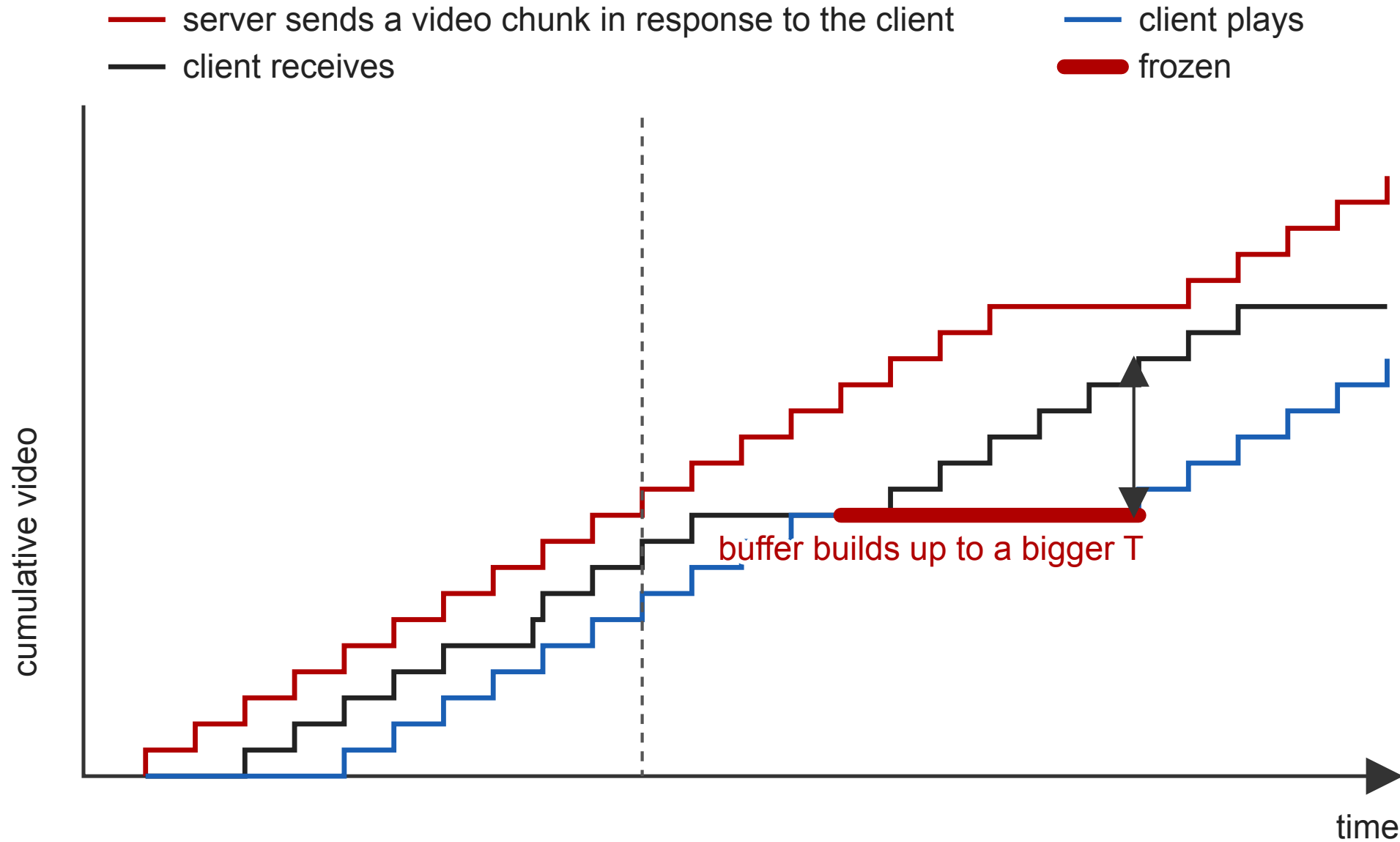
A longer pipe: use a bigger T



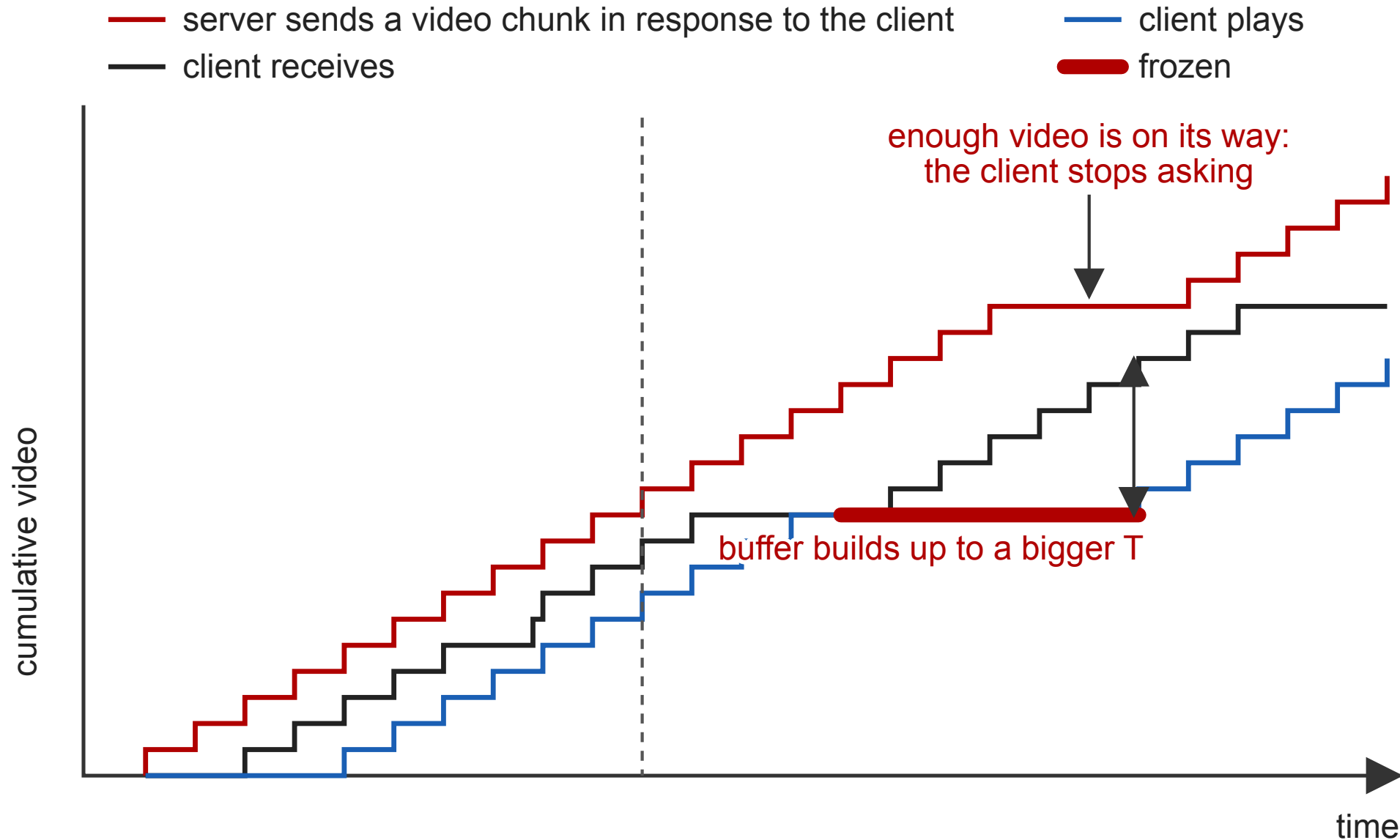
A longer pipe: use a bigger T



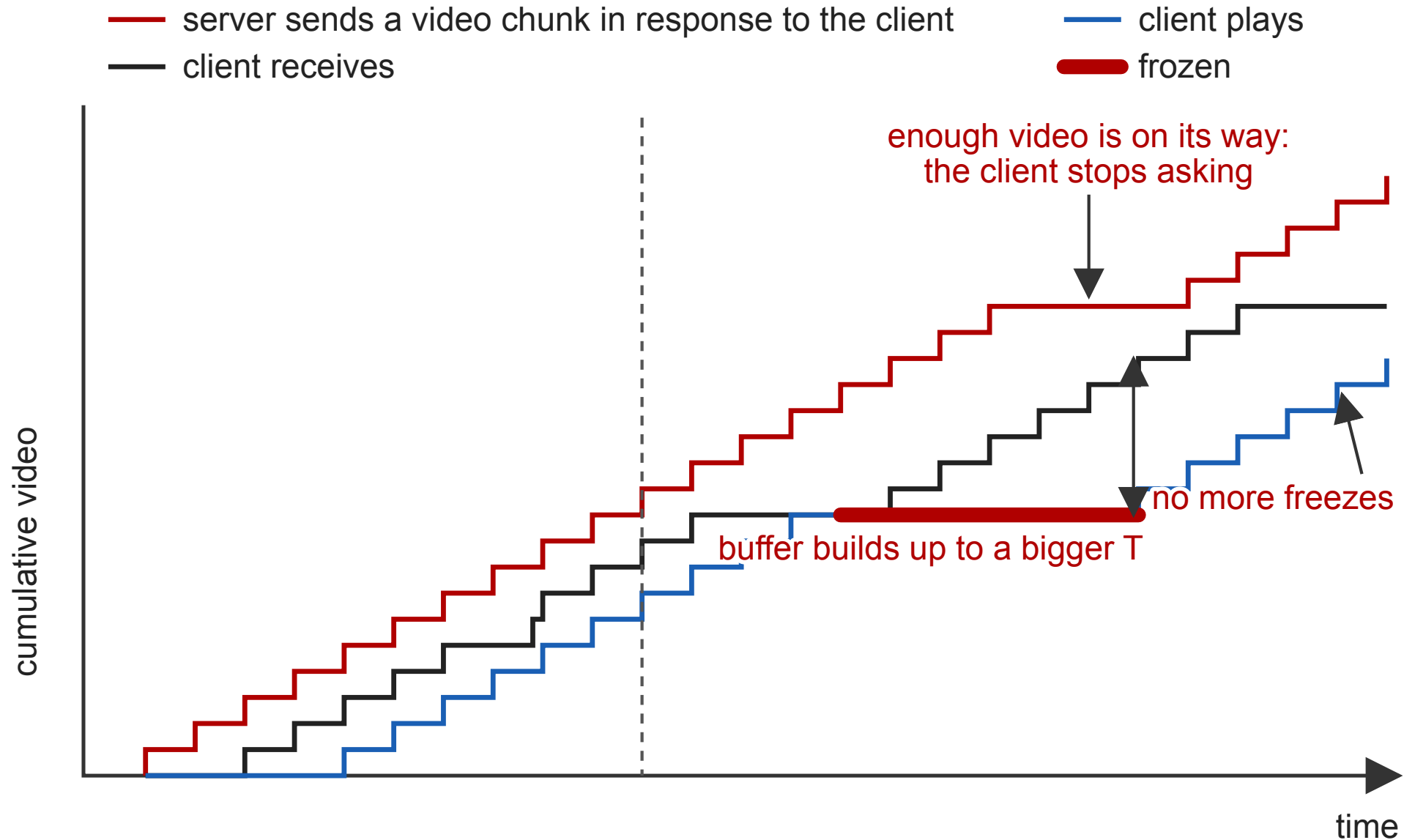
A longer pipe: use a bigger T



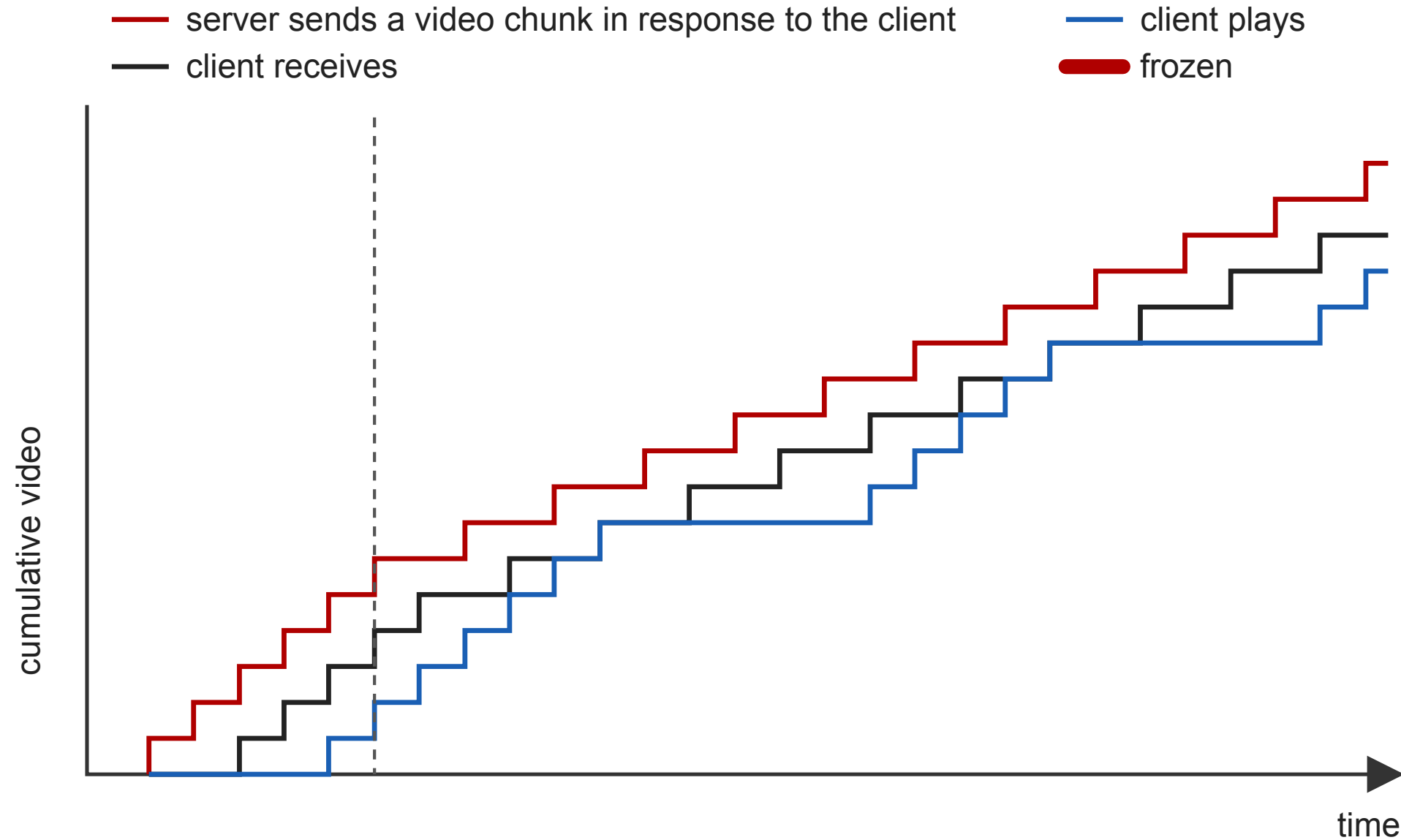
A longer pipe: use a bigger T



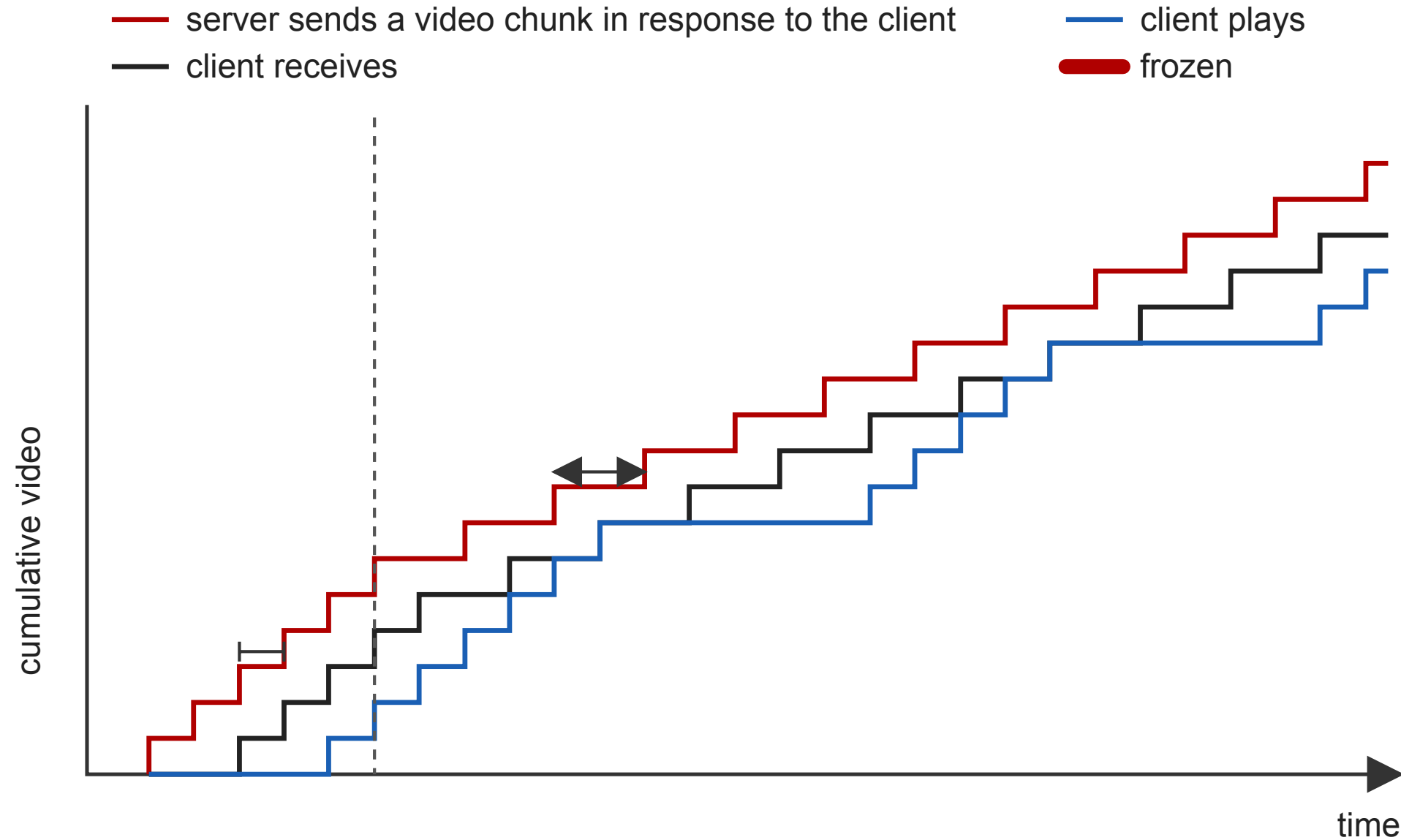
A longer pipe: use a bigger T



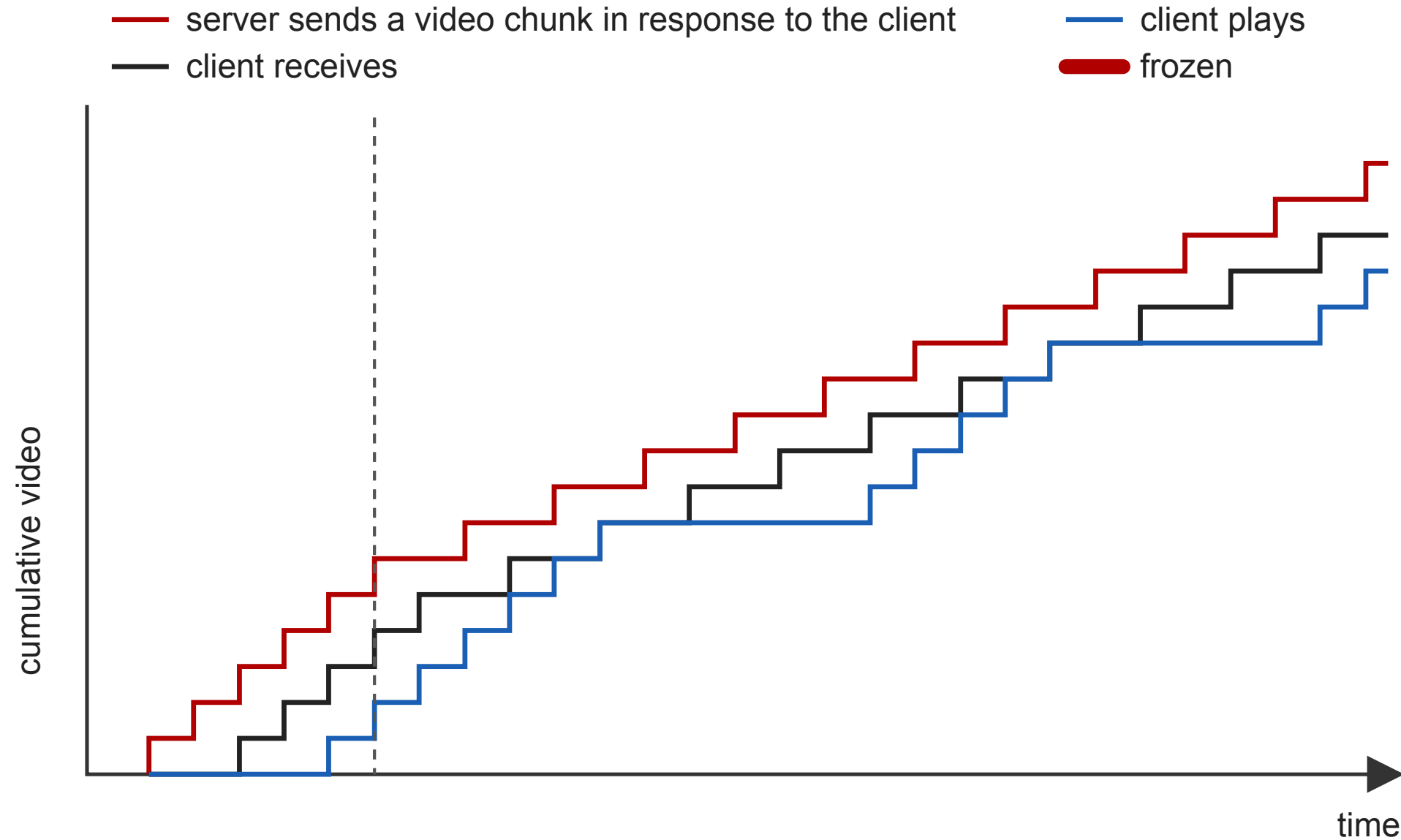
If the pipe gets narrower for a while



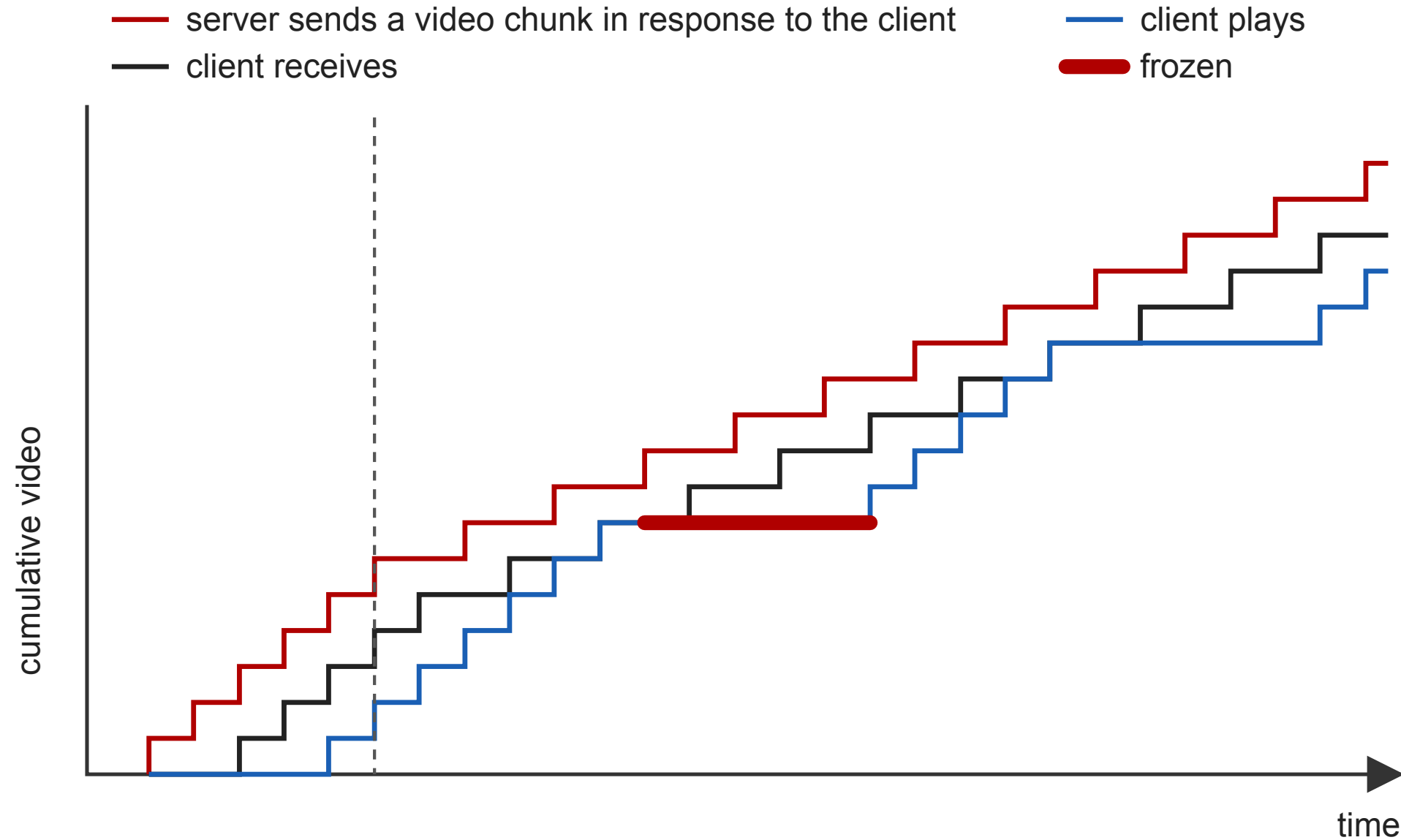
If the pipe gets narrower for a while



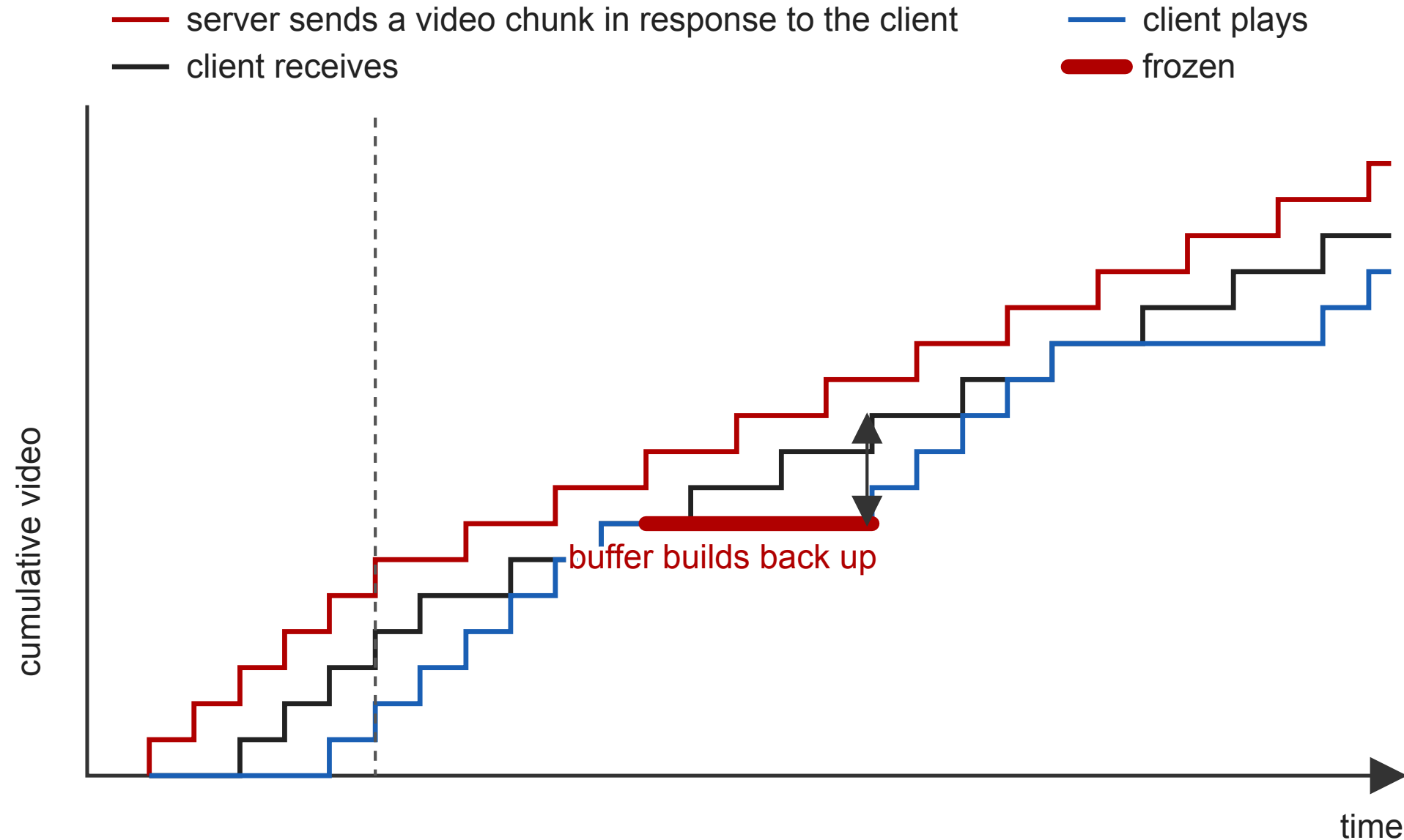
If the pipe gets narrower for a while



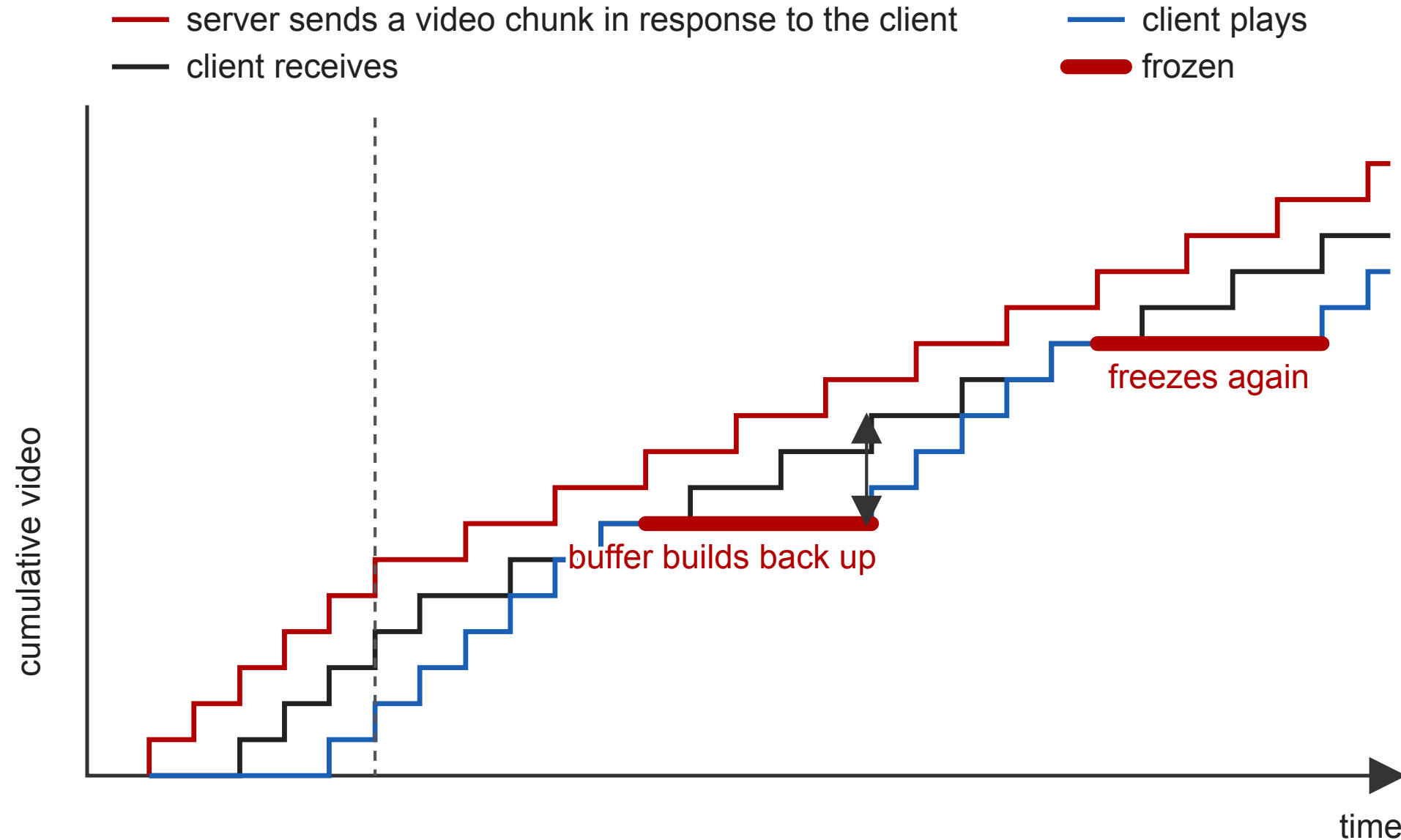
If the pipe gets narrower for a while



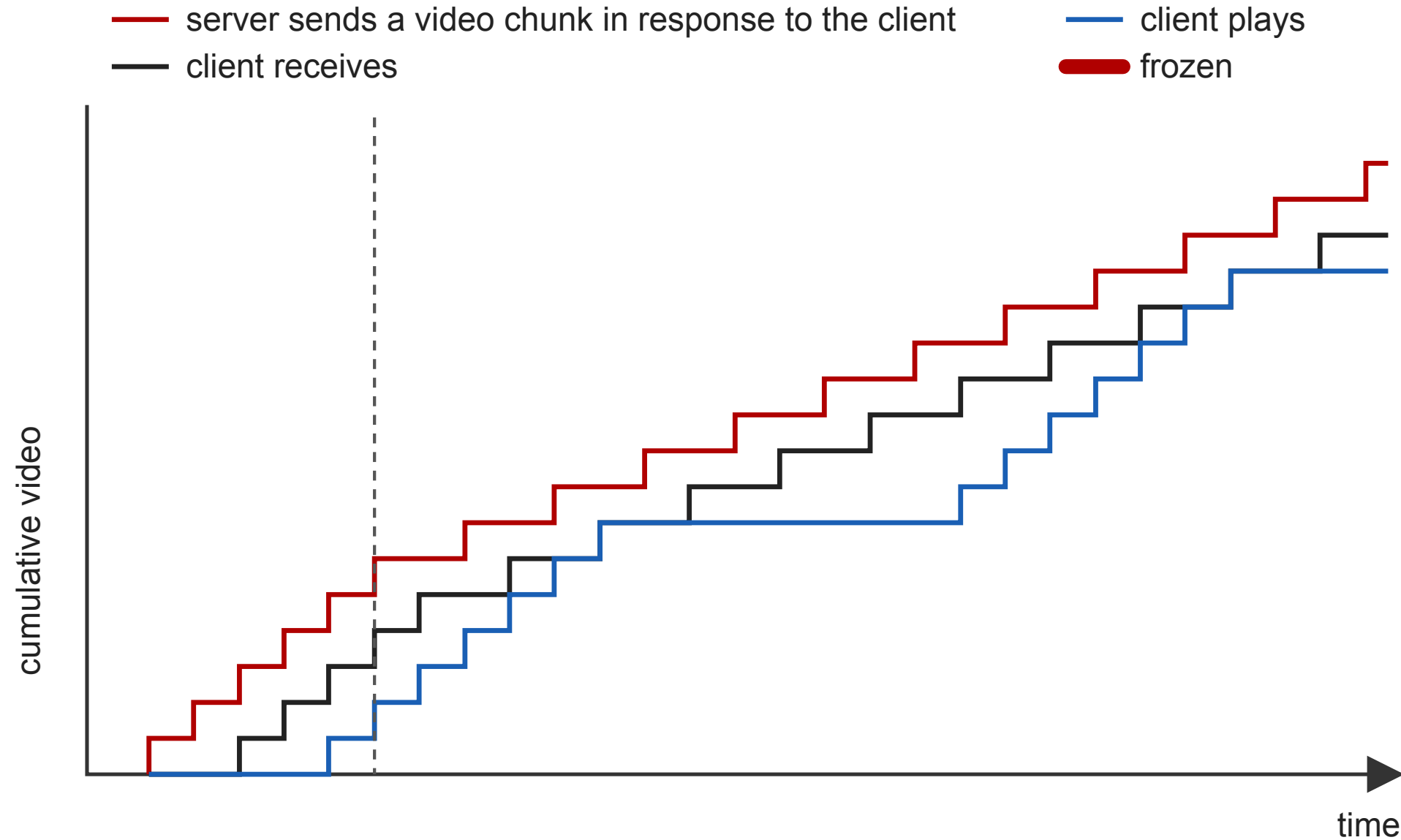
If the pipe gets narrower for a while



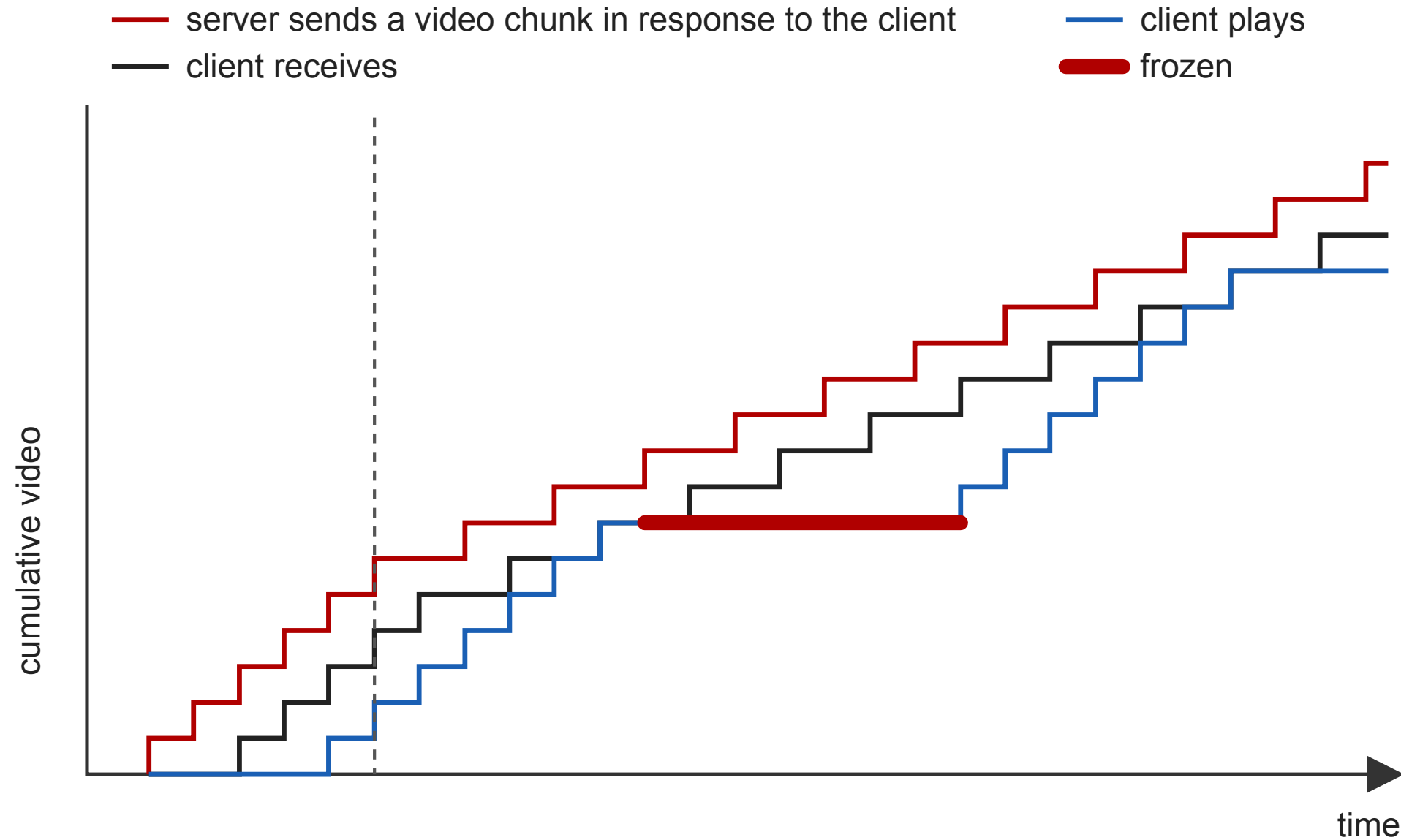
If the pipe gets narrower for a while



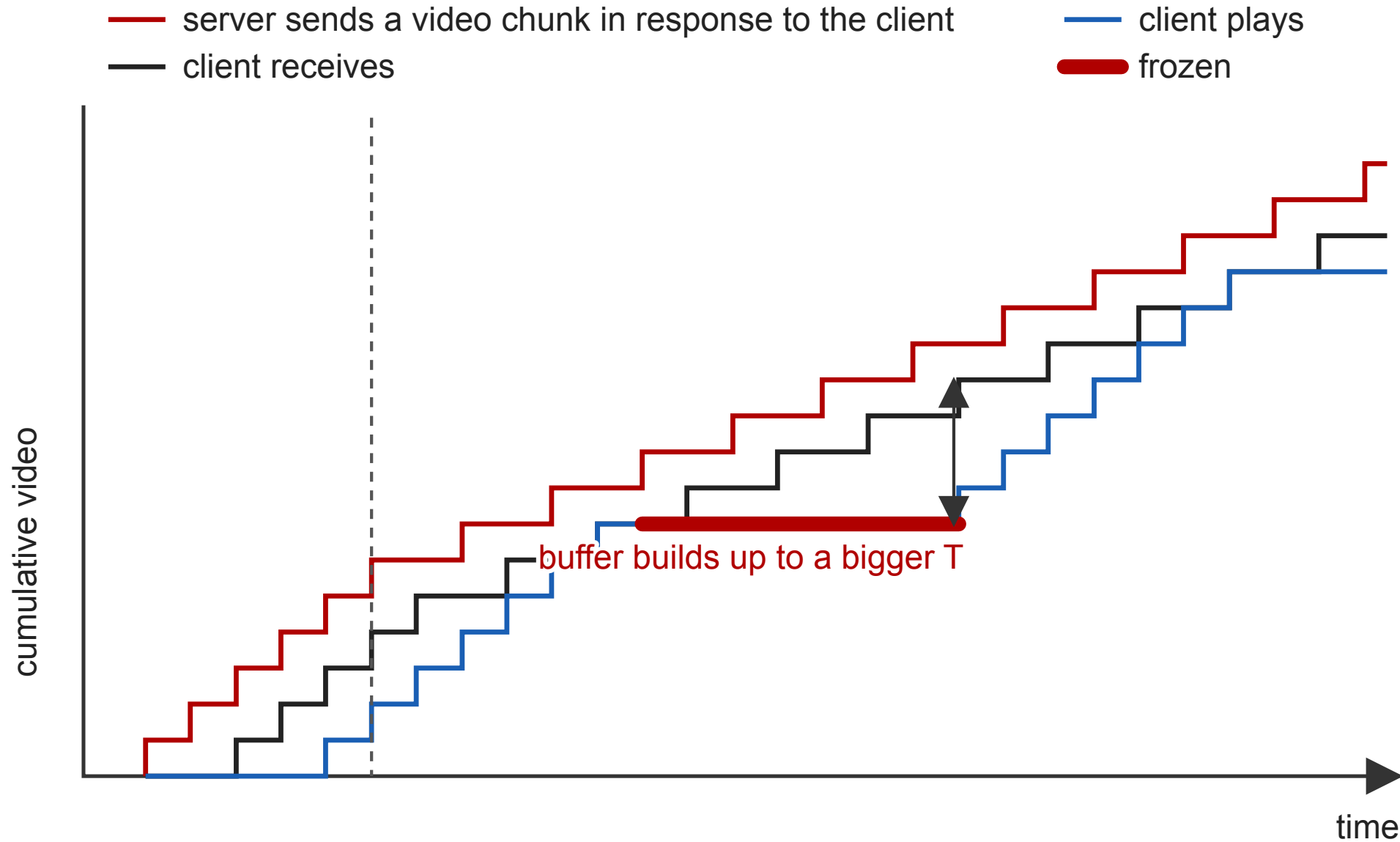
Would a bigger T help?



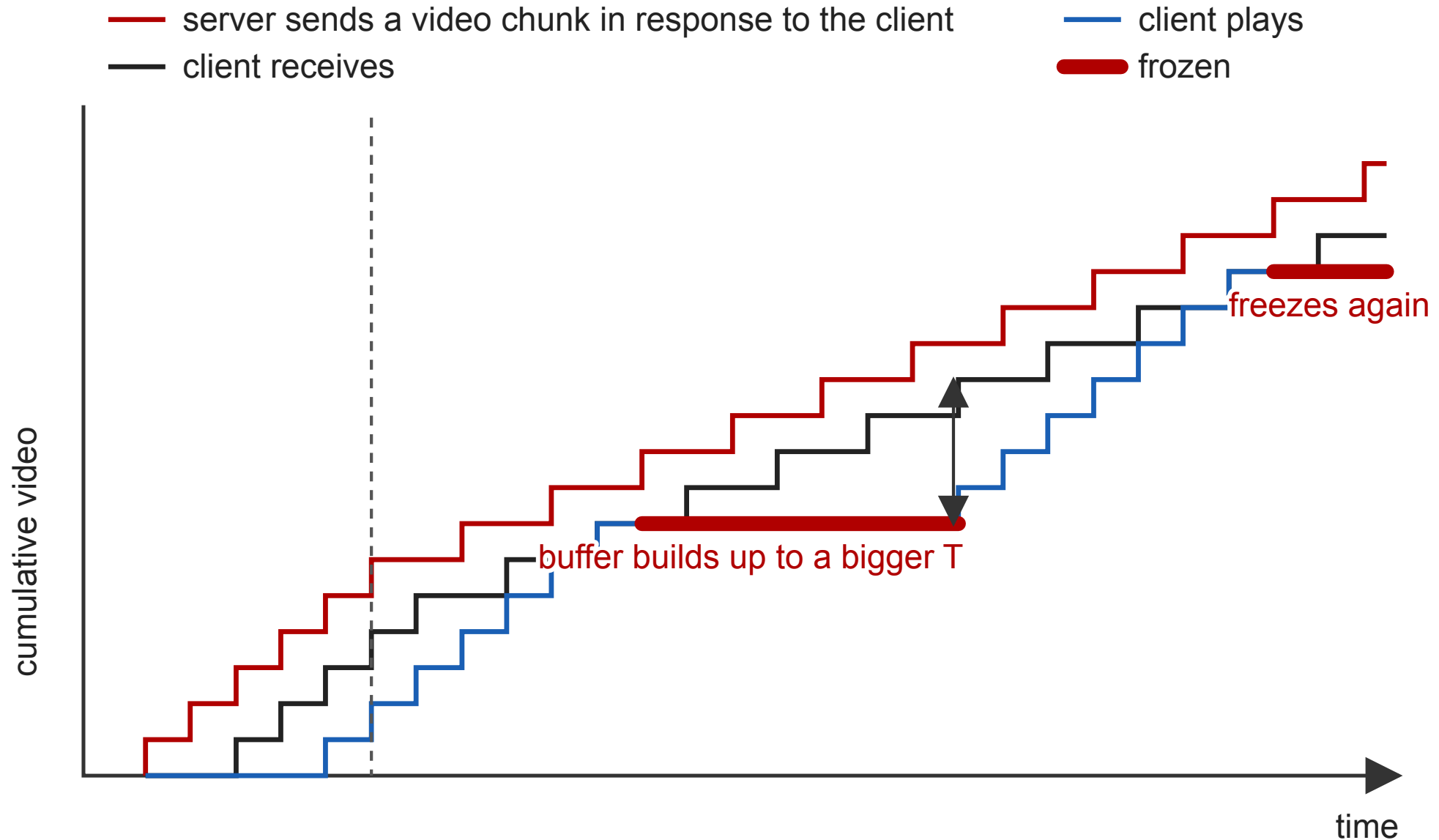
Would a bigger T help?



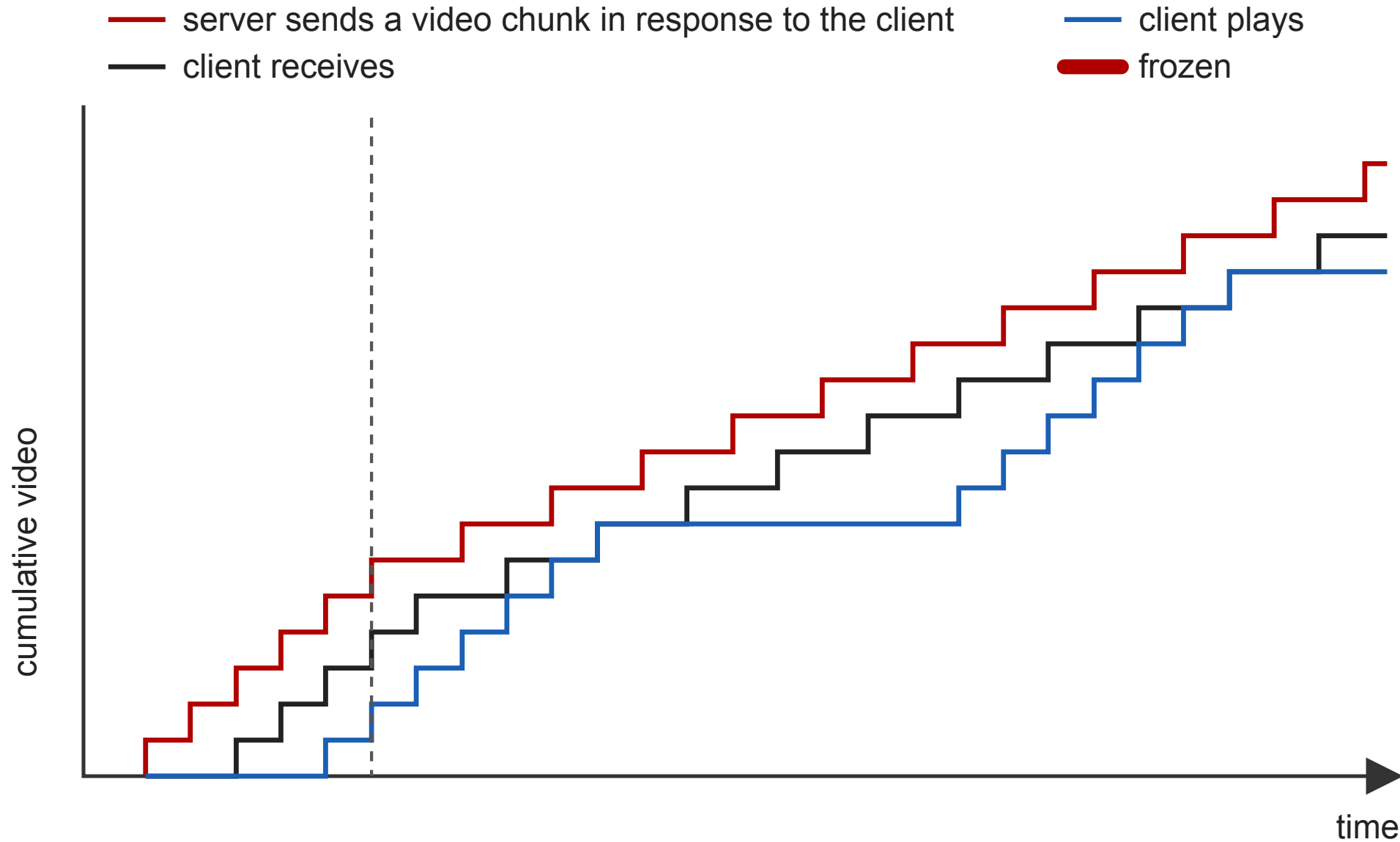
Would a bigger T help?



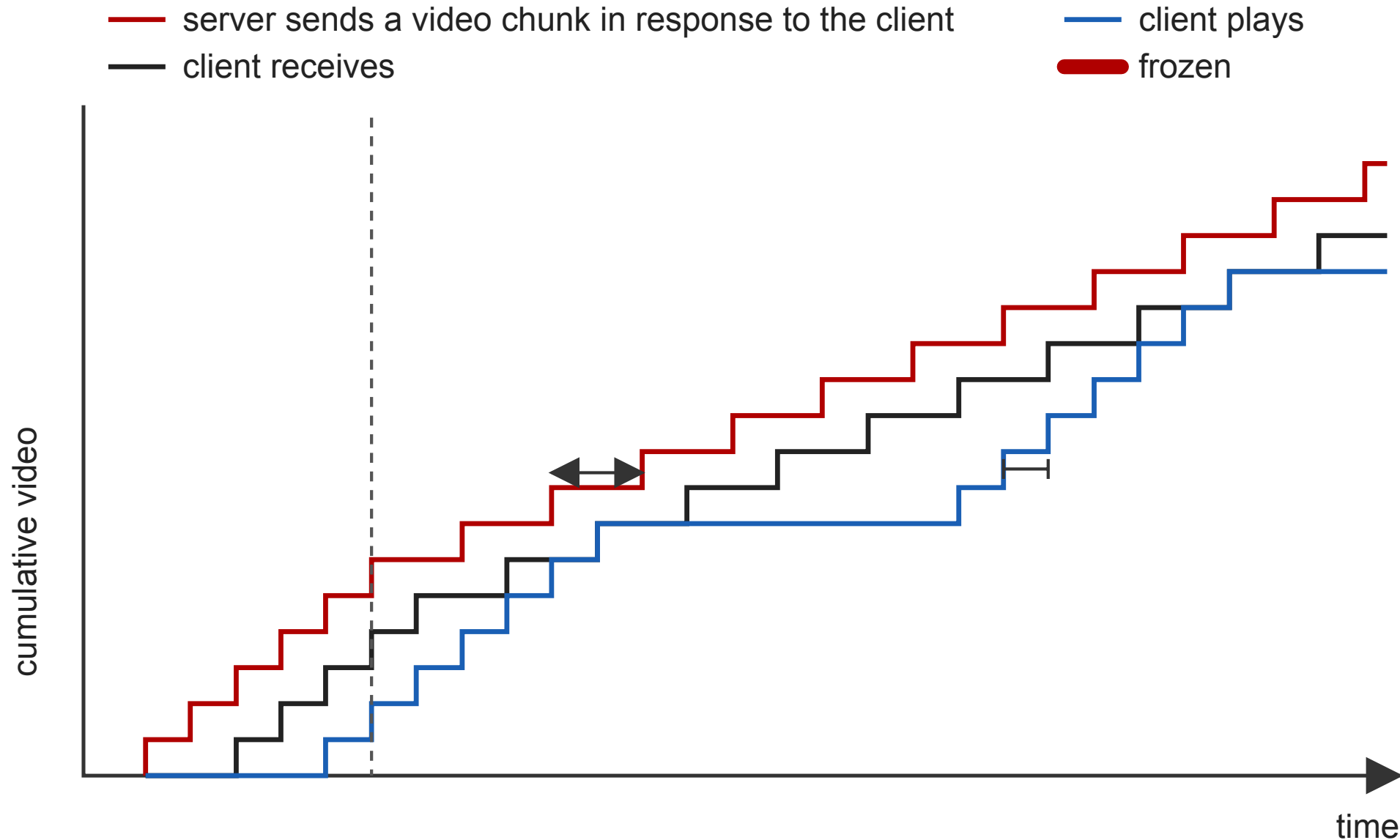
Would a bigger T help?



A chunk takes longer to send than to play



A chunk takes longer to send than to play



Ask for smaller chunks

- The server keeps every chunk at several qualities: low, medium, high
- A lower-quality chunk is still 2 s of video, but fewer bytes: it arrives sooner
- The client monitors its playback buffer:
 - draining: ask for the next chunk at a lower quality
 - comfortably full: ask for a higher quality again

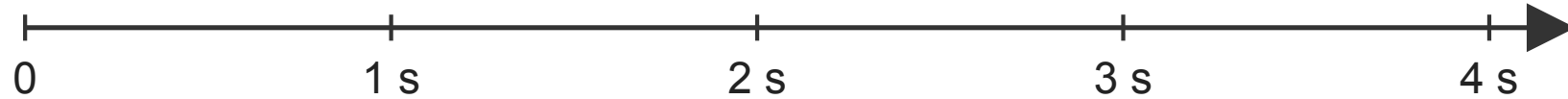
Ask for smaller chunks

- The server keeps every chunk at several qualities: low, medium, high
- A lower-quality chunk is still 2 s of video, but fewer bytes: it arrives sooner
- The client monitors its playback buffer:
 - draining: ask for the next chunk at a lower quality
 - comfortably full: ask for a higher quality again

DASH (Dynamic Adaptive Streaming over HTTP)

The same 2 s of video, fewer bytes

60 frames, shown one every $\frac{1}{30}$ s: 2 s of video

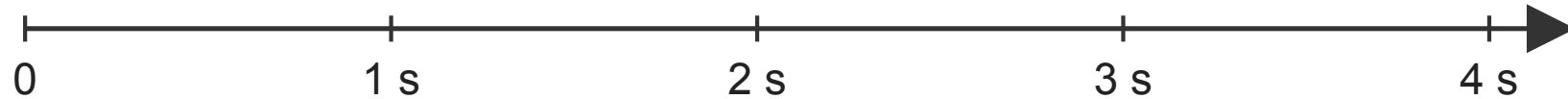


The same 2 s of video, fewer bytes

60 frames, shown one every $\frac{1}{30}$ s: 2 s of video



high quality, over the narrower pipe: they arrive over 4 s



The same 2 s of video, fewer bytes

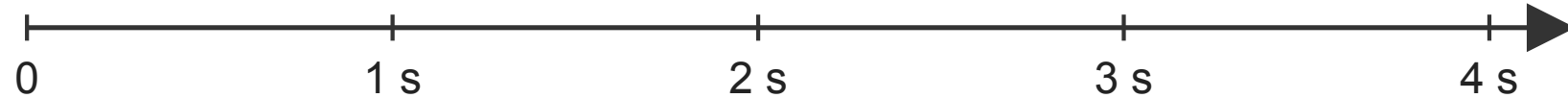
60 frames, shown one every $\frac{1}{30}$ s: 2 s of video



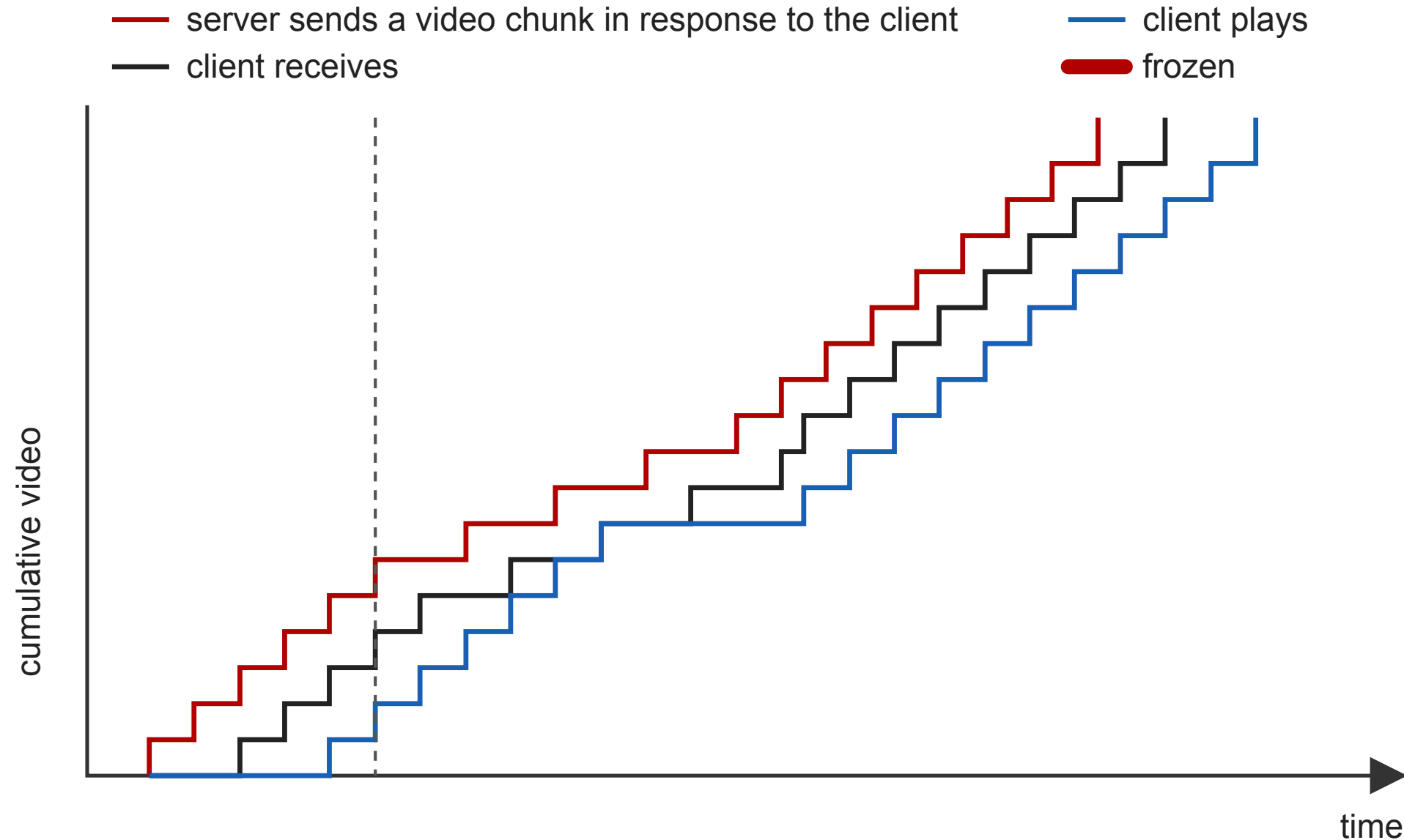
high quality, over the narrower pipe: they arrive over 4 s



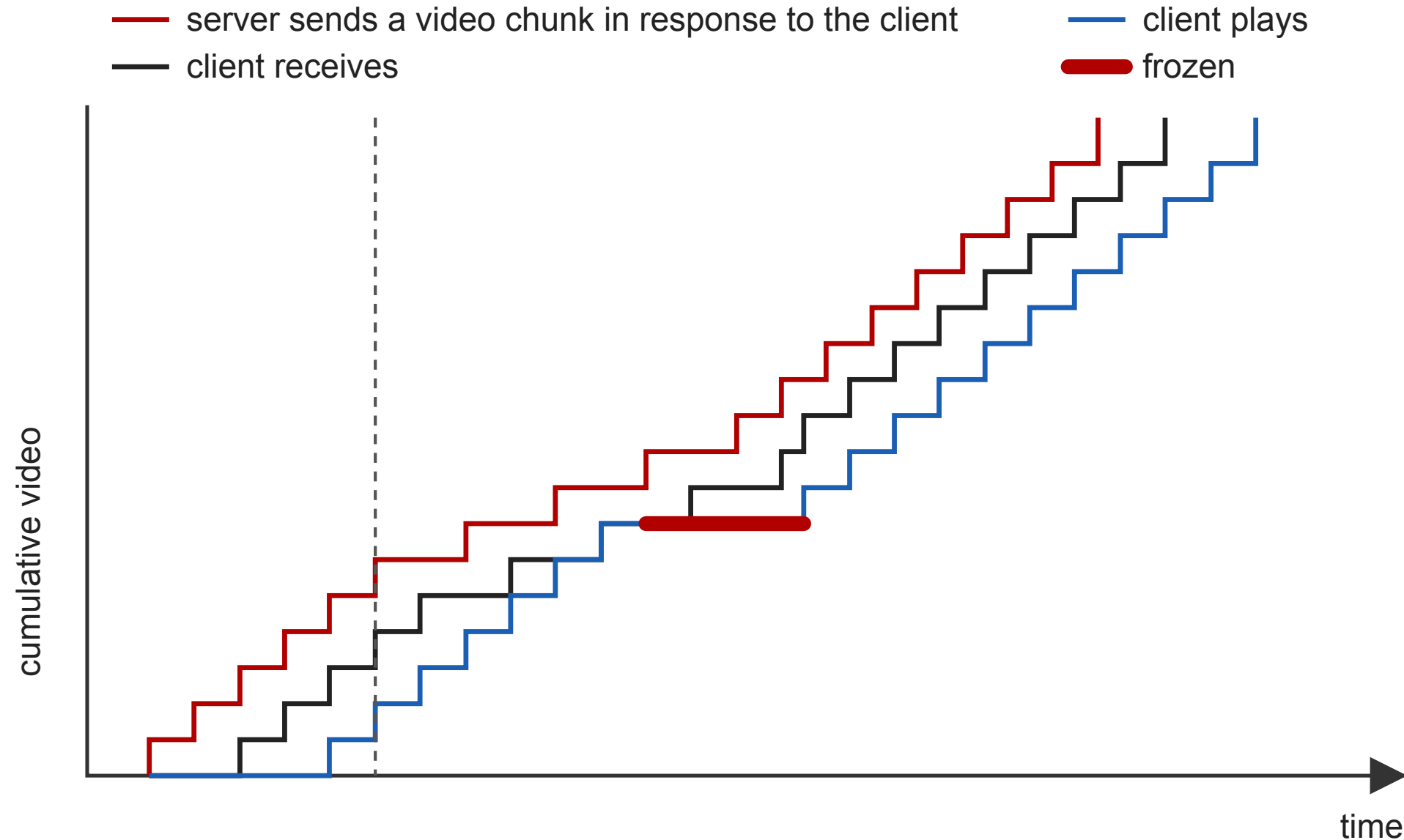
lower quality, same frames with fewer bytes: they arrive in 1 s



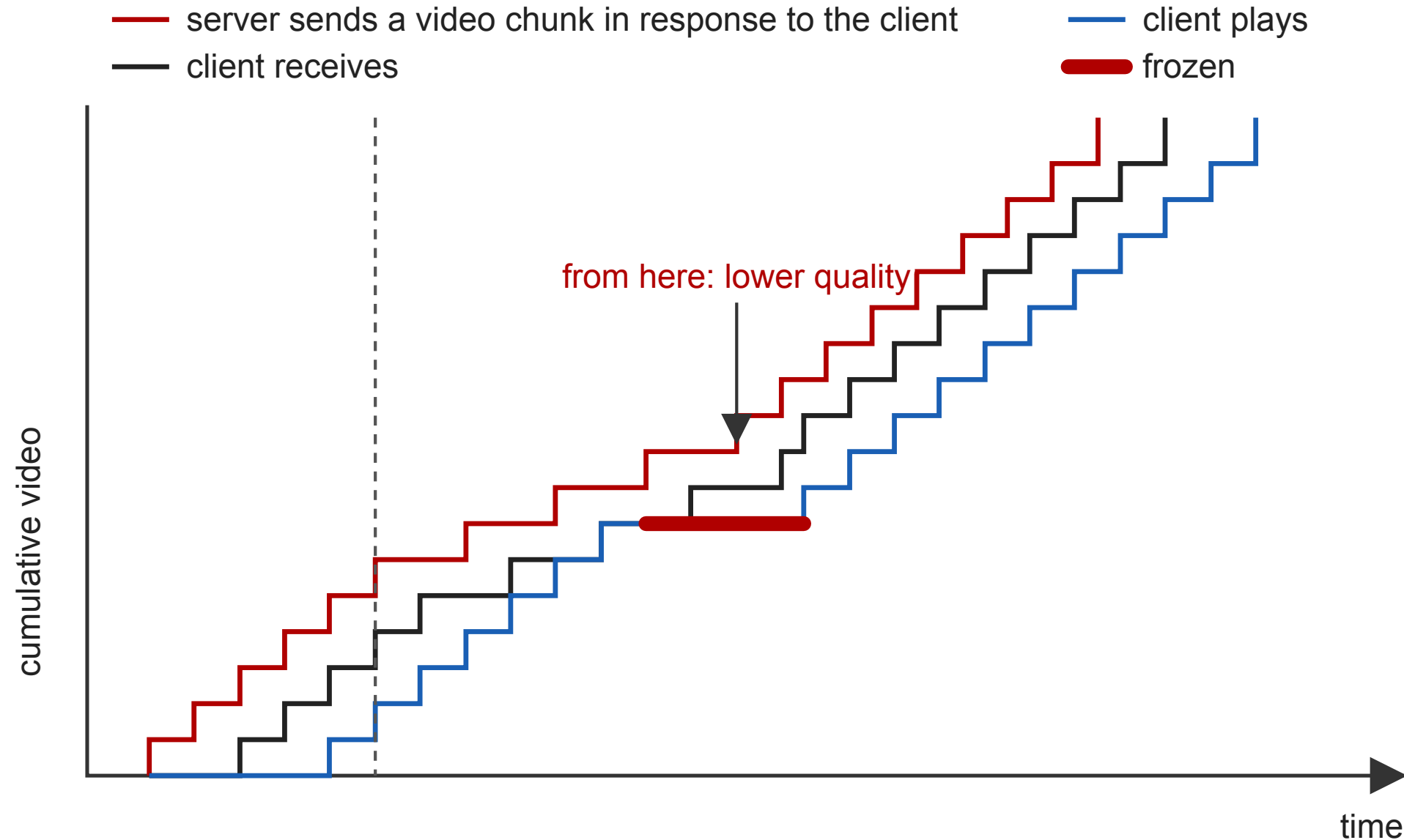
If the client switches to a lower quality



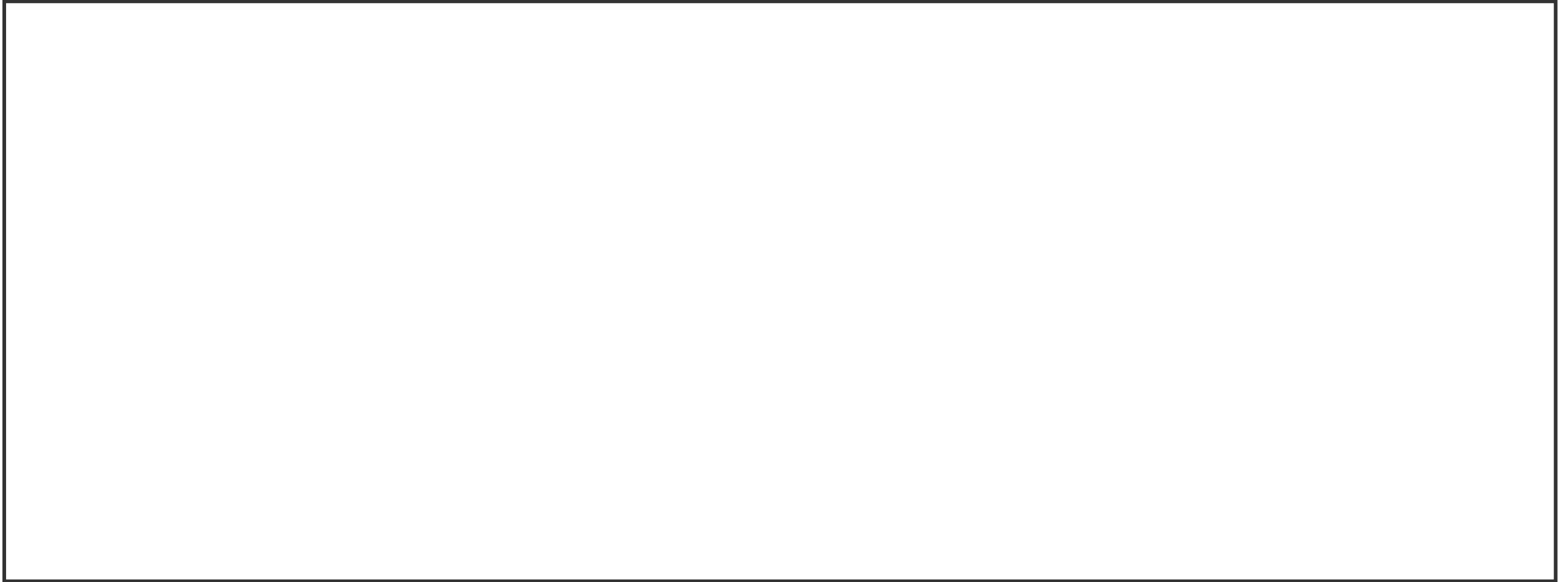
If the client switches to a lower quality



If the client switches to a lower quality



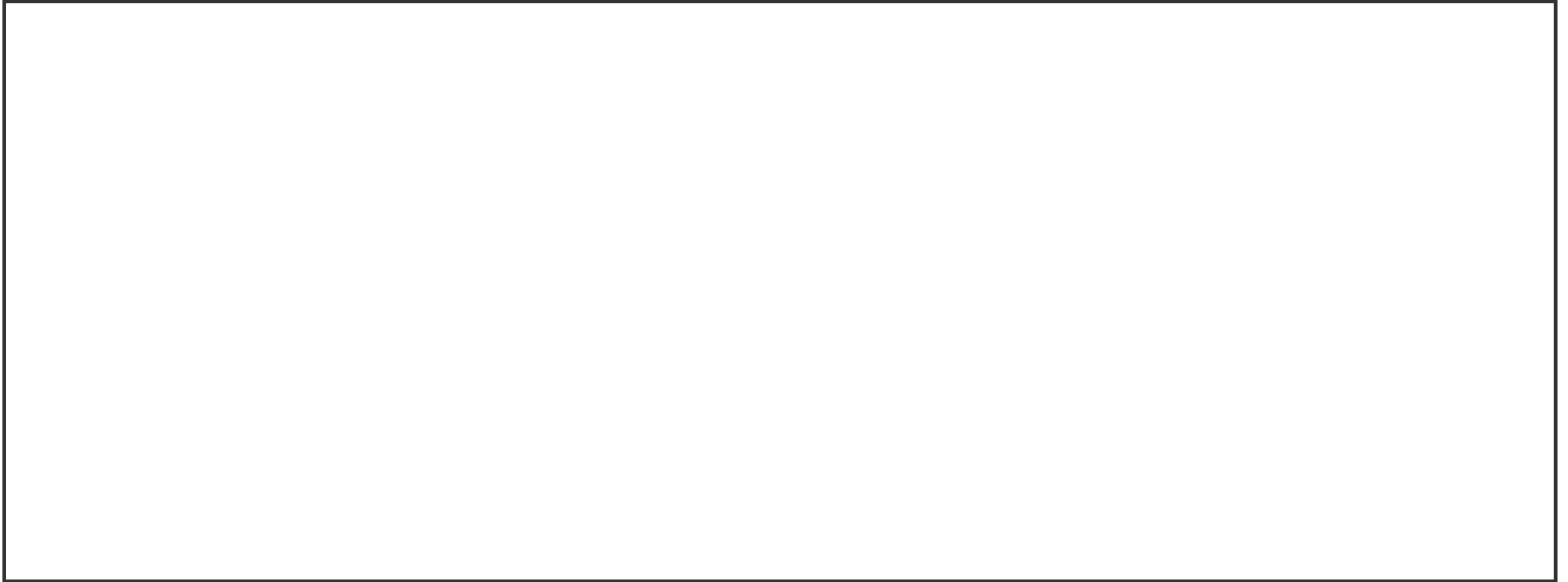
What if this was a video call instead of on-demand video streaming?



Outline

- Where we were
- Video streaming on a perfect network
- Video streaming on a real network
- **What each application wants from the network**
- Takeaways

What metrics and properties matter to an application?



What do applications want from the network?

	Web browser	Video streaming	Video call